mediatools 1.2.12 – Developer Documentation

Architecture, UML, dynamic behaviour, data formats, quality gates and API reference

Peter Wein

2026-11-09

Abstract

This document is written for developers who maintain or extend mediatools. It describes the repository layout, the coding standard (Google-style docstrings enforced by pylint's docparams extension, maximum line length 100, pylint score 10.00/10 for source, tests and tools), the layered architecture with one class per module, UML class diagrams of every package generated from the code, sequence diagrams of shrink, triage, flatten and sync, the activity diagram of a snapshot run, the local configuration file, state diagrams of a player process and of a flat link, activity diagrams of the decision logic, all persistent data formats, error handling and exit codes, the testing strategy with its test doubles, the five quality gates (format, lint, types, security, tests), the git-flow branching model and the release procedure, and how to add a subcommand. The appendix contains the complete API reference, generated from the docstrings by `tools/gen_docs.py`.

Contents

1 Purpose and audience **2**

2 Repository layout **2**

3 Coding standard **3**
3.1 Rules . . . 3
3.2 Docstring example . . . 3
3.3 pylint configuration . . . 3

4 Architecture **4**
4.1 Layers . . . 4
4.2 Package `core` . . . 5
4.3 Package `shrink` . . . 7
4.4 Package `triage` . . . 9
4.5 Package `flatten` . . . 10
4.6 Package `sync` . . . 11
4.7 Package `snapshot` . . . 12
4.8 Package `update` . . . 13
4.9 Package `cli` . . . 14

5 Dynamic behaviour **15**
5.1 shrink . . . 15
5.2 triage . . . 17
5.3 flatten . . . 18
5.4 sync . . . 19
5.5 snapshot . . . 21

6 Data formats **22**

7 Error handling, signals, exit codes **22**

8 Testing **23**
8.1 Strategy . . . 23
8.2 Coverage of the safety rules . . . 23

9 Quality gates **23**

10 Branching model **25**
10.1 Branch protection . . . 26
10.2 Feature cycle . . . 26

11 Building the documentation **26**

12 Release procedure **26**

13 Applying delivered patch archives **27**

14 Adding a subcommand **28**

15 Glossary **28**

16 Sources **29**

17 Appendix: API reference **30**
17.1 `mediatools.cli` . . . 30
17.2 `mediatools.core` . . . 34
17.3 `mediatools.shrink` . . . 56
17.4 `mediatools.triage` . . . 78
17.5 `mediatools.flatten` . . . 87
17.6 `mediatools.sync` . . . 94
17.7 `mediatools.snapshot` . . . 102
17.8 `mediatools.update` . . . 112

1 Purpose and audience

The user handbook (`docs/handbook.md`) explains *how to use* mediatools. This document explains *how it is built*: which class does what, how the classes interact at run time, which files they read and write, and which rules every change has to follow.

2 Repository layout

```
mediatools/
  README.md                project page (GitHub)
  CHANGELOG.md             release history
  CONTRIBUTING.md          rules for contributions
  LICENSE                  MIT
  pyproject.toml           packaging, pytest and pylint configuration
  install.sh               installs a release tag into a venv (~/scripts/mediatools)
  .github/workflows/       CI: tools/check.sh on Python 3.10, 3.12, 3.13
  .githooks/pre-push       runs tools/check.sh before every branch push
  src/mediatools/          package (one class per module)
  tests/                   pytest suite
  tools/check.sh           quality gates
  tools/gen_docs.py        UML class diagrams + API reference from the code
  docs/handbook.md         user handbook (+ PDF)
  docs/developer.md        this document (+ PDF)
  docs/api.md              generated API reference
  docs/img/                Mermaid sources (*.mmd) and rendered PNG
```

3 Coding standard

3.1 Rules

1. **One class per module.** The module name describes the class (`shrink/database.py` contains `ShrinkDatabase`).
2. **Google-style docstrings** on every module, class, method and function, including private ones: summary line, `Args:`, `Returns:` or `Yields:`, `Raises:` where applicable; classes list their instance variables in `Attributes:`.
3. **Comments** explain *why* a block exists wherever this is not obvious from the code (e.g. "writes into the existing inode", "in-place assignment prunes the walk").
4. **pylint 10.00/10** for `src`, `tests` and `tools`; maximum line length 100. The configuration lives in `pyproject.toml`.
5. **Standard library only.** External programs (ffmpeg, ffprobe, rsync, gio, players) are called with argument lists, never through a shell.
6. **Dependency injection.** Logger, prompt, probe, player, remover and file-system helpers are constructor arguments, so tests can replace them.
7. **Plan before action** for numerous or irreversible changes: analyse, summarise, confirm, execute.

3.2 Docstring example

```python
def name(self, parts: list[str]) -> tuple[str, int] | None:
    """Build the flat name for a relative path.

    Args:
        parts: Path components below the unit; the last element is the
            file name.

    Returns:
        ``(flat_name, level)``, or None if no level fits the limit.
    """
```

3.3 pylint configuration

```toml
[tool.pylint.main]
load-plugins = ["pylint.extensions.docparams", "pylint.extensions.docstyle"]

[tool.pylint.format]
max-line-length = 100

[tool.pylint.parameter_documentation]
default-docstring-type = "google"
accept-no-param-doc = false
accept-no-raise-doc = false
accept-no-return-doc = false
accept-no-yields-doc = false

[tool.pylint.design]
max-args = 10                 # collaborators are injected (testability)
max-positional-arguments = 10
max-attributes = 12
max-locals = 22
min-public-methods = 1
```

The design limits are raised deliberately: constructors receive their collaborators as arguments instead of creating them, which costs arguments but makes every class testable in isolation. The only remaining inline suppression is `consider-using-with` for the long-lived player processes in `PlayerSession.open`, which must outlive the method.

4 Architecture

4.1 Layers

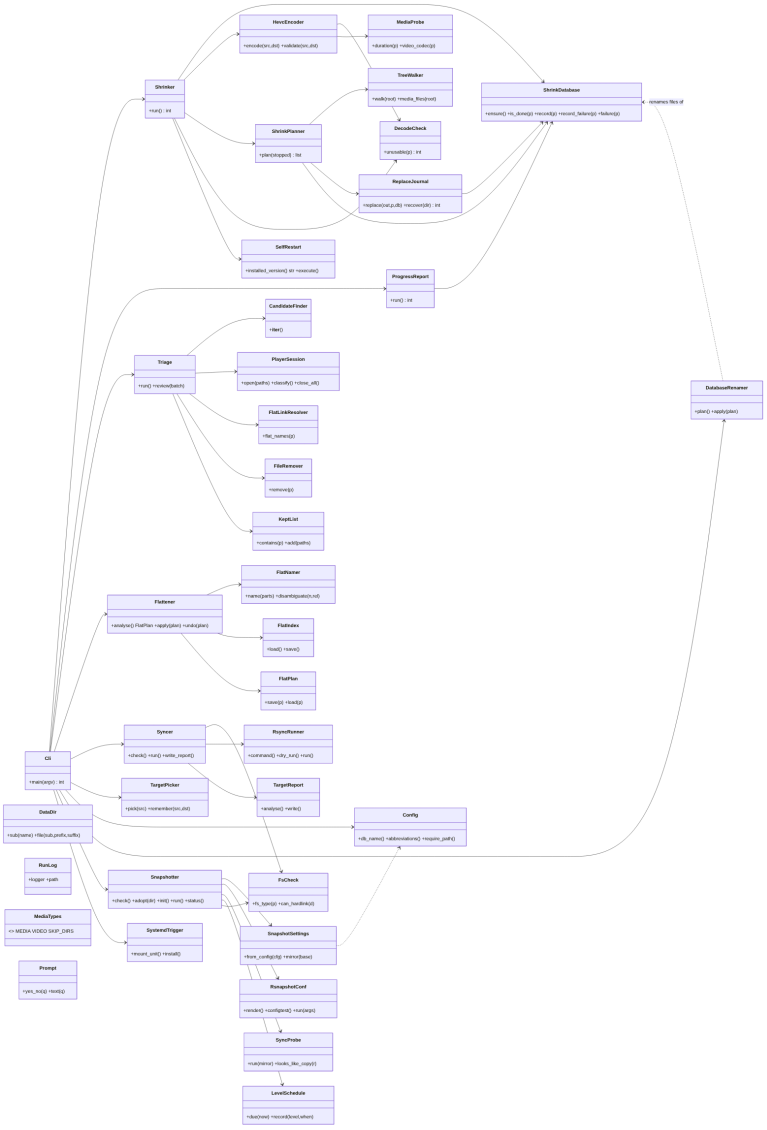


Figure 1: All classes and their dependencies

Layer	Package	May import
interface	`mediatools.cli` (`Cli`)	all packages
features	`shrink`, `triage`, `flatten`, `sync`, `snapshot`	`core`; `snapshot` also `sync.fscheck`
foundation	`core`	standard library only

The feature packages do not import each other, with one exception: `snapshot` reuses `sync.FsCheck` (file-system type, free space, nesting). Shared knowledge – reserved directory names, media extensions, data directory, configuration – lives in `core`. `triage` cooperates with `flatten` only through the file system (inodes in `_flat`), not through imports.

4.2 Package core

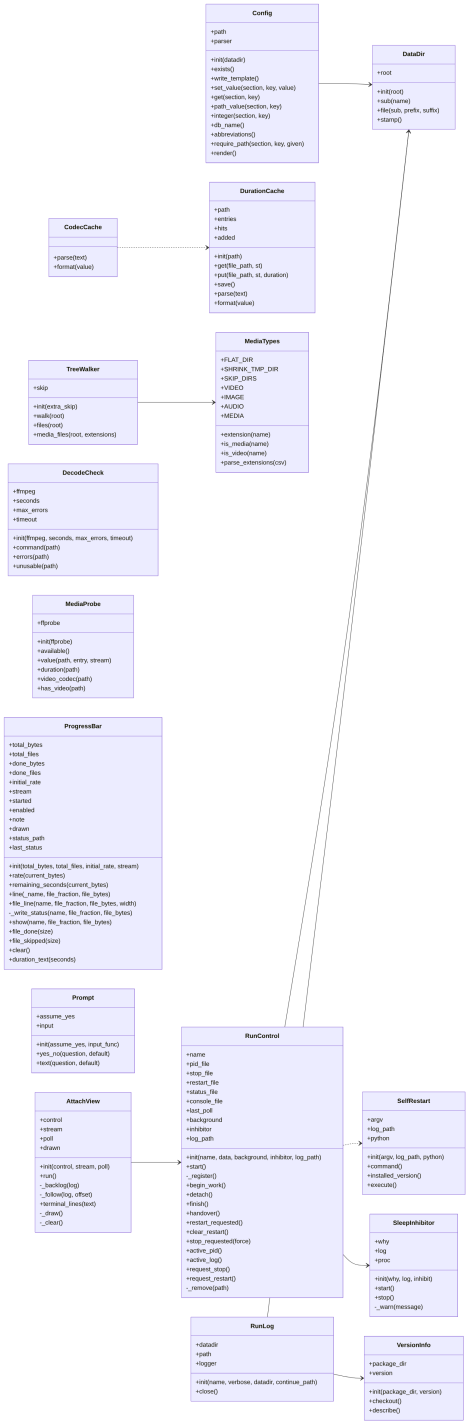


Figure 2: core

Class	Responsibility
`MediaTypes`	extensions, `_flat` / `newShrinkedFiles` names, `SKIP_DIRS`

Class	Responsibility
`Config`	local INI file: collection root, database name, abbreviations, snapshot settings; error messages name key and expected format
`DataDir`	`$MEDIATOOLS_HOME` layout: logs, state, plans, reports
`RunLog`	logger to terminal and time-stamped file
`MediaProbe`	ffprobe: duration, codec, video stream
`SelfRestart`	switch of a running shrink to the installed release: `--version` probe, command line (`--foreground -y --continue-log`), `os.execv` (same PID)
`DecodeCheck`	`verify()`: full decode of an output, any error line or timeout = failure; `unusable()`: trial decode of the first 10 s (`ffmpeg -t 10 -i FILE -f null -`); 10 or more error lines = data not decodable; no verdict if ffmpeg is missing or times out
`DurationCache`	ffprobe durations keyed by path, size and mtime (shared by triage and shrink)
`ProgressBar`	two-line terminal display (run and current file), pending stop, status file for `attach`
`RunControl`	PID/stop/restart/status files of a running job, refuses a second live job of the same name, hands its registration over on a release switch, background mode (fork, `setsid`), sleep lock in the working process
`AttachView`	`mediatools attach`: log backlog, new lines, progress lines; Ctrl+C leaves the view
`SleepInhibitor`	systemd sleep/idle inhibitor lock for the lifetime of a job
`TreeWalker`	sorted `os.walk` that never enters reserved/hidden dirs
`Prompt`	yes/no and text questions, `--yes` support

4.3 Package `shrink`

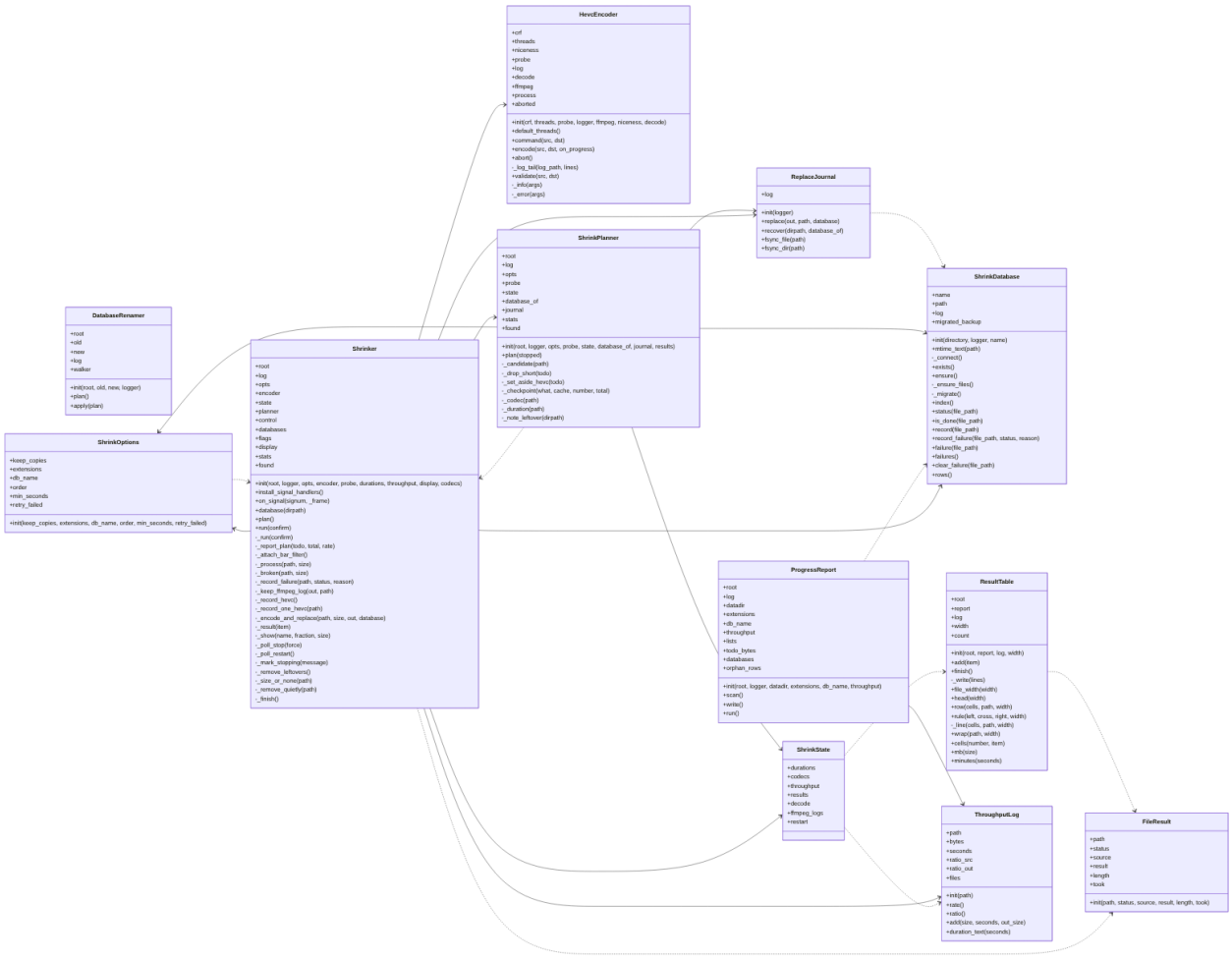


Figure 3: shrink

Class	Responsibility
`ShrinkDatabase`	database per directory (name configurable), matching by file name, schema migration, UPSERT
`DatabaseRenamer`	one-time renaming of databases and their backups, never overwrites
`HevcEncoder`	ffmpeg in its own process group, progress, abort, validation of exit code, size (at least 2 % of the input), stream, duration and full decode of the output (`DecodeCheck.verify`); message file kept on failure
`ShrinkPlanner`	planning phase: recovery, leftovers, database and failure rows, regrown originals (larger than recorded, not HEVC: planned again), minimum duration, HEVC set-aside, order
`ReplaceJournal`	crash-safe in-place copy-back with marker file, recovery after a crash

Class	Responsibility
`Shrinker`	per-file decision (HEVC, trial decode, encode), failure rows, kept ffmpeg messages, two-stage Ctrl+C, stop request via `RunControl`, summary
`ProgressReport`	read-only done/changed/todo lists, orphan rows, time estimate
`ShrinkOptions`	settings of a run (order, minimum duration, review mode, DB name)
`ThroughputLog`	measured encoding speed and size ratio for estimates
`ShrinkState`	typed collection of the run helpers (caches, throughput, result table, trial decode)
`FileResult`	outcome of one video (status, sizes, times)
`ResultTable`	framed per-file result table: terminal, log, `reports/shrink_results_<ts>.txt`

4.4 Package `triage`

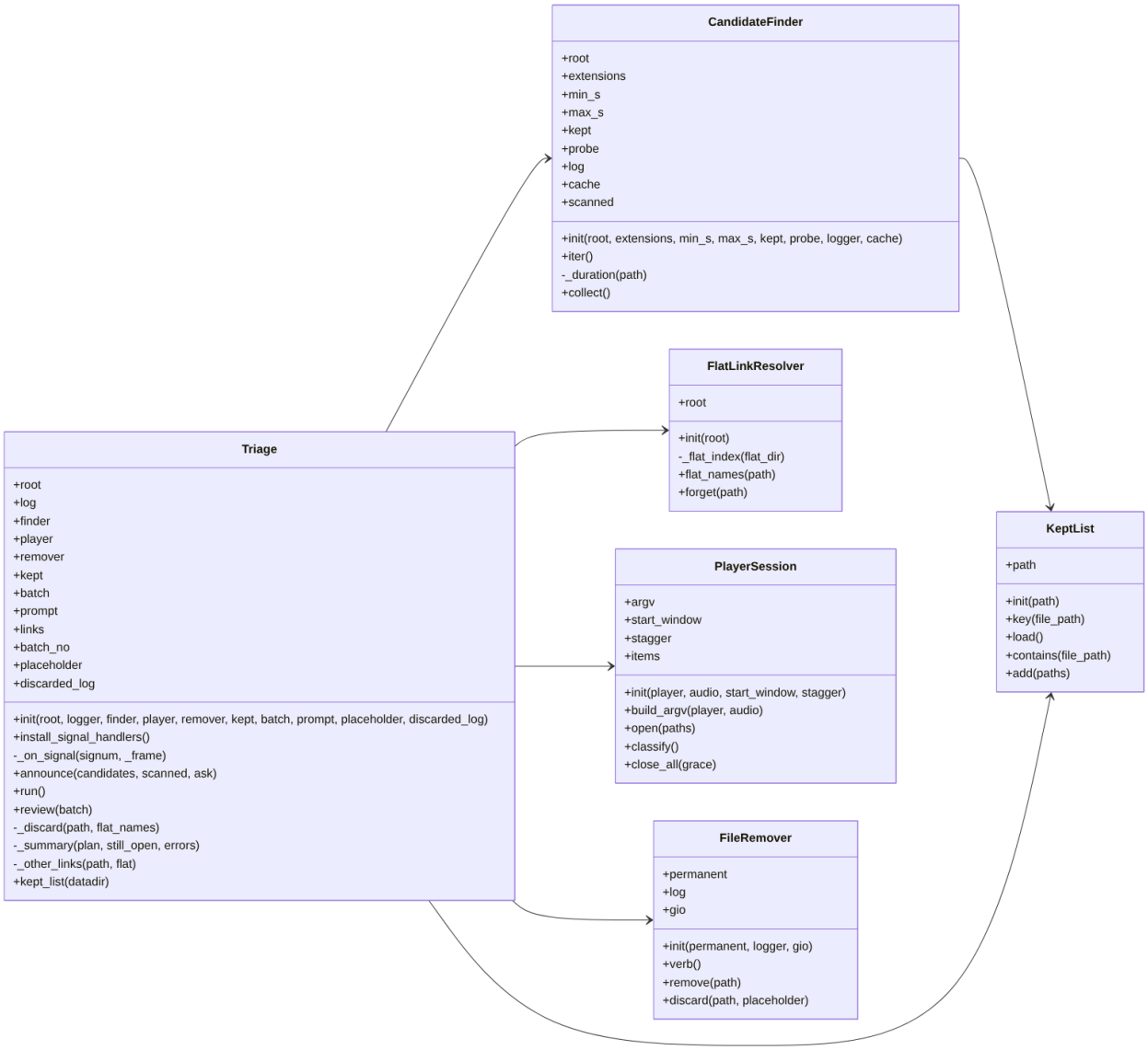


Figure 4: triage

Class	Responsibility
`CandidateFinder`	search of videos within a duration range; `collect()` counts first
`PlayerSession`	one player process per video, classification
`FlatLinkResolver`	`_flat` names of a file by inode
`FileRemover`	`gio trash` or permanent deletion; `discard()` leaves a 0-byte placeholder
`KeptList`	decisions across sessions
`Triage`	batch workflow

4.5 Package `flatten`

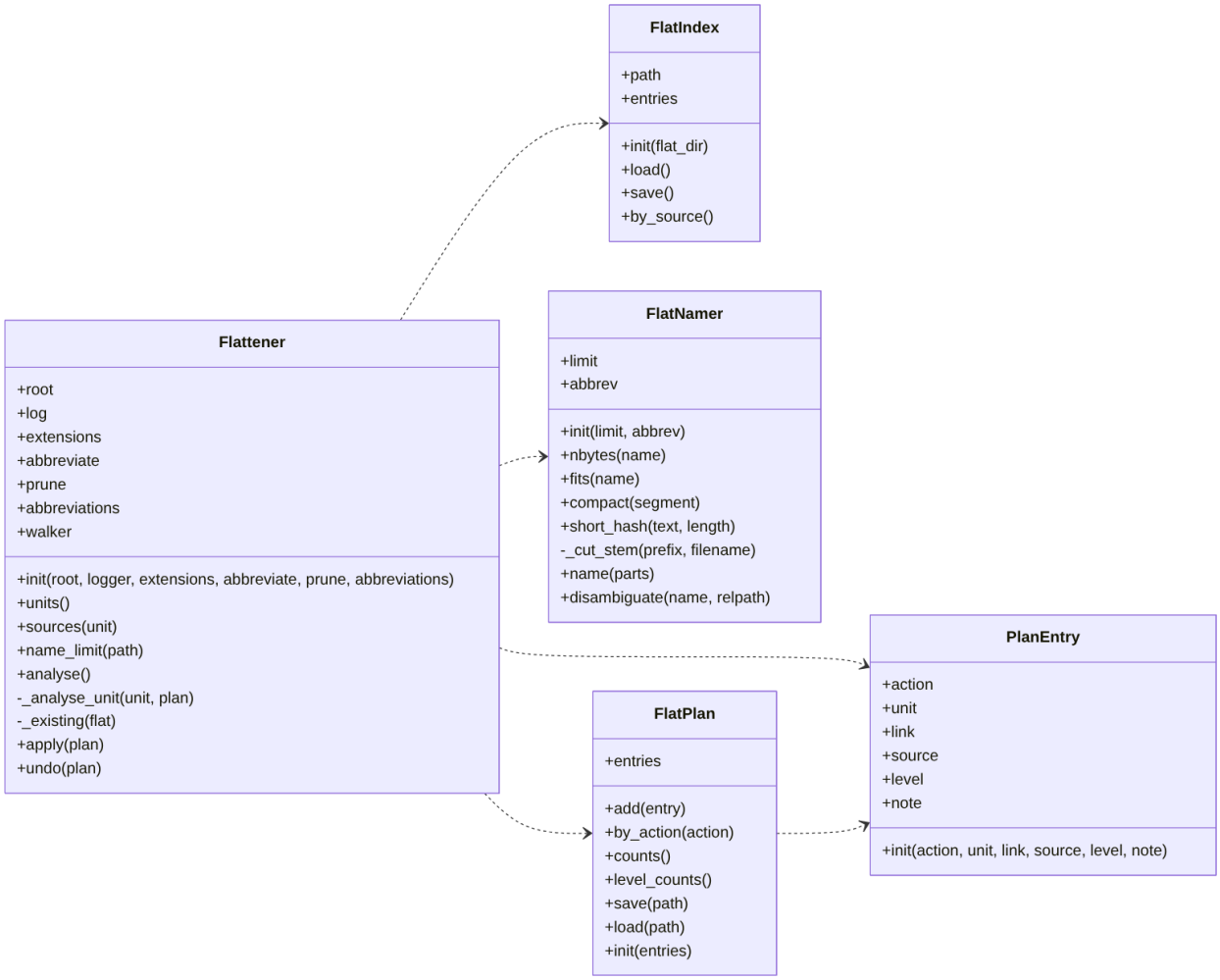


Figure 5: flatten

Class	Responsibility
`FlatNamer`	names within the byte limit, levels 0..3, disambiguation
`FlatIndex`	`_flat/_index.tsv`, atomic save
`PlanEntry`	one planned or executed flatten action
`FlatPlan`	actions, counts, TSV persistence
`Flattener`	discovery, analysis, apply, undo

4.6 Package `sync`

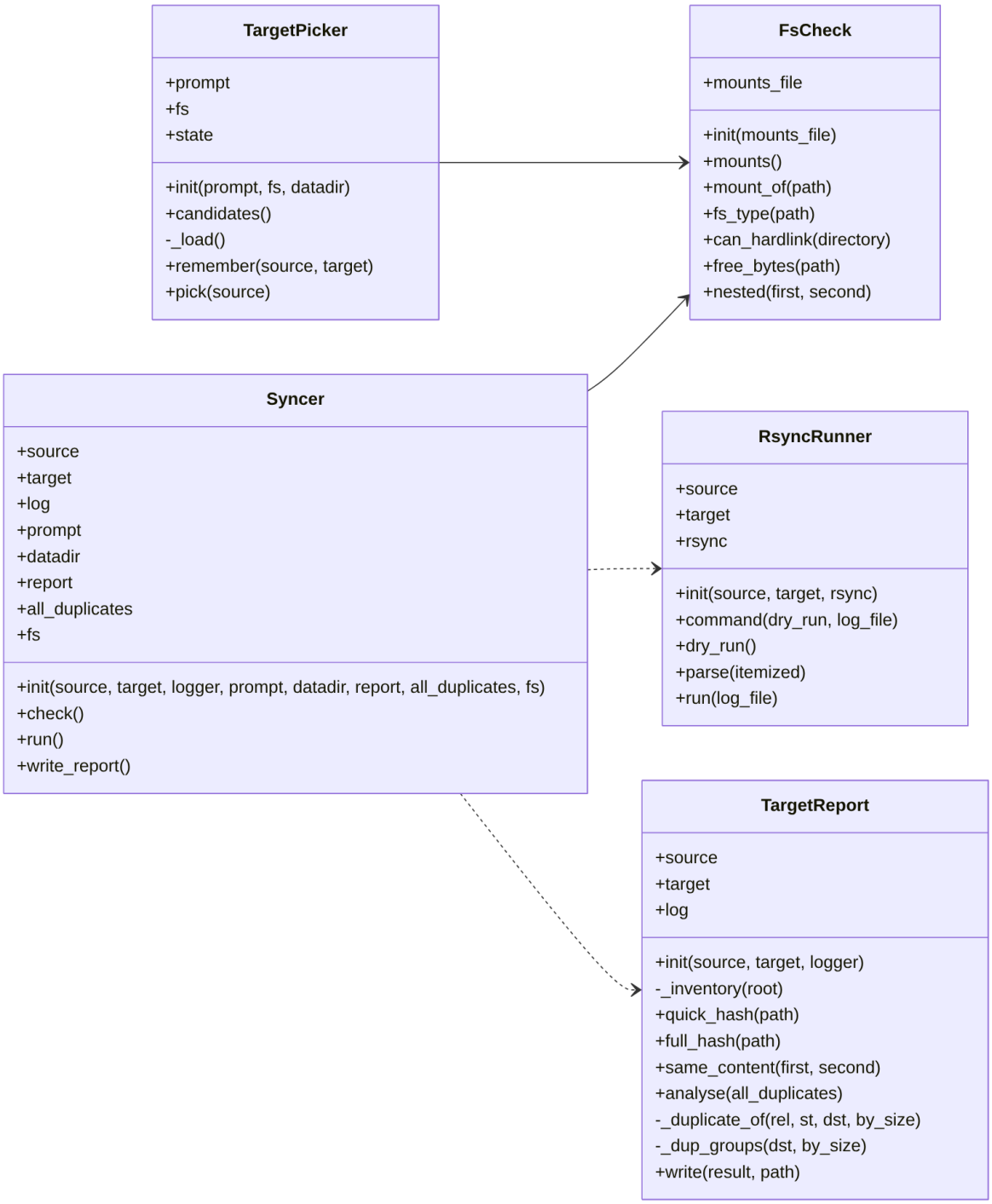


Figure 6: sync

Class	Responsibility
`FsCheck`	mount table, fs type, practical hard-link test, nesting
`TargetPicker`	interactive disk choice, remembered per source
`RsyncRunner`	rsync command, dry-run parsing, real run

Class	Responsibility
`TargetReport`	target-only files, duplicate detection by hash
`Syncer`	checks, dry run, confirmation, transfer, report

4.7 Package `snapshot`

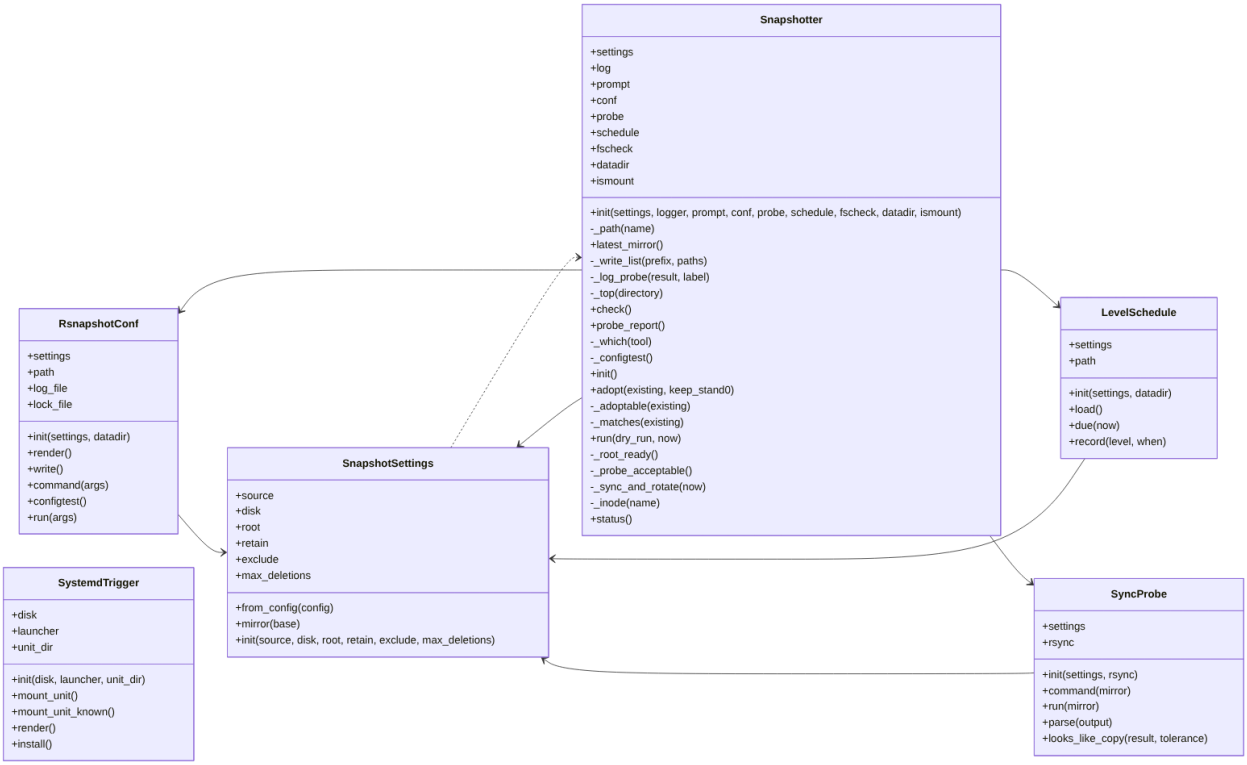


Figure 7: snapshot

Class	Responsibility
`SnapshotSettings`	validated `[snapshot]` values, mirror path inside a snapshot
`RsnapshotConf`	generated `rsnapshot.conf`, `configtest`, rsnapshot calls
`SyncProbe`	rsync dry run: transfer volume, removals, plausibility
`LevelSchedule`	which higher levels are due, state file
`Snapshotter`	check, adopt, init, run, status
`SystemdTrigger`	user unit bound to the disk’s mount unit

4.8 Package `update`



Figure 8: update

Class	Responsibility
`ReleaseUpdater`	`mediatools update`: runs the bundled `apply-release.sh` (see “Applying delivered patch archives”)

4.9 Package `cli`



Figure 9: cli

`Cli` builds one `argparse` sub-parser per subcommand; `main()` dispatches to `cmd_<name>(args)`, which creates the objects of that feature, wires them and returns the exit code.

Arrows in the class diagrams: `-->` the class receives or creates the collaborator in its constructor; `..>` the class uses it inside a method. The diagrams are generated by `tools/gen_docs.py` from the code, so they cannot drift from the implementation.

5 Dynamic behaviour

5.1 shrink

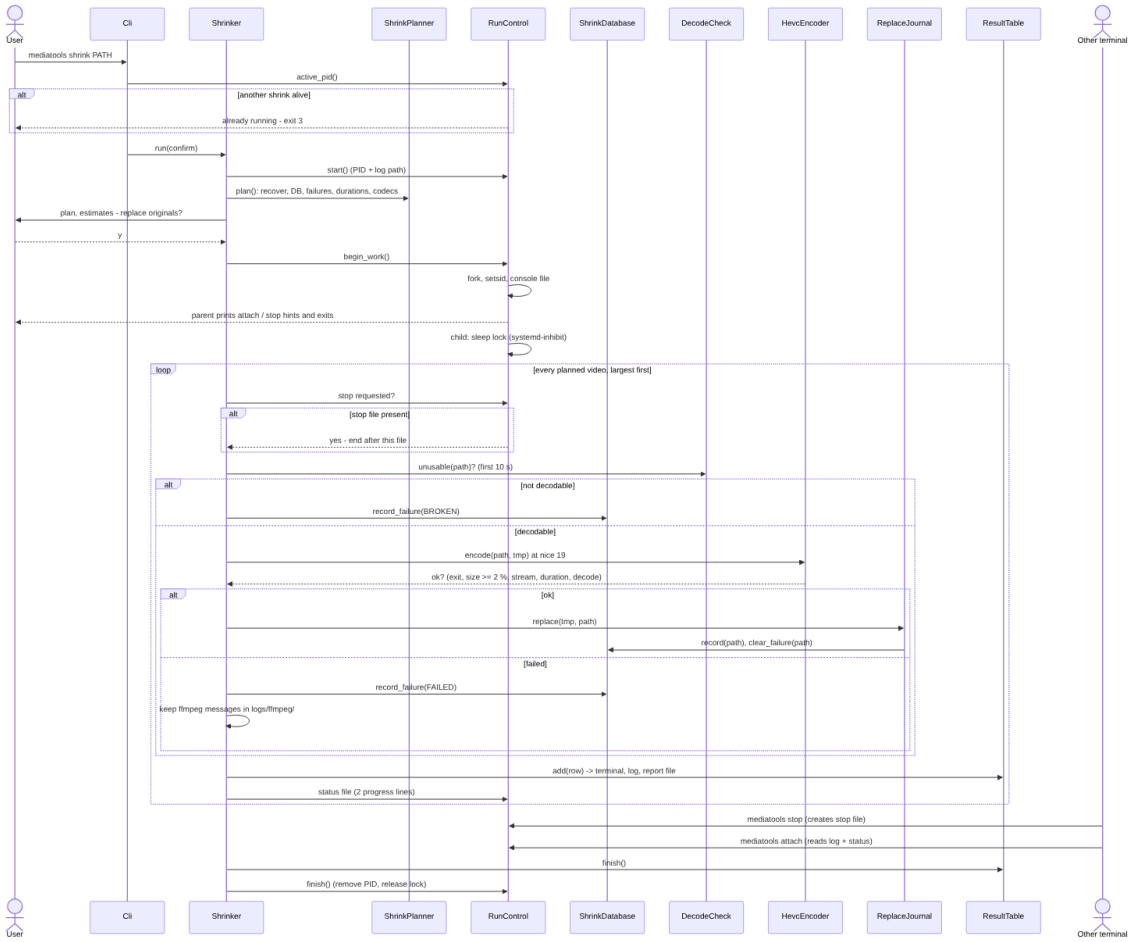


Figure 10: Sequence of a shrink run

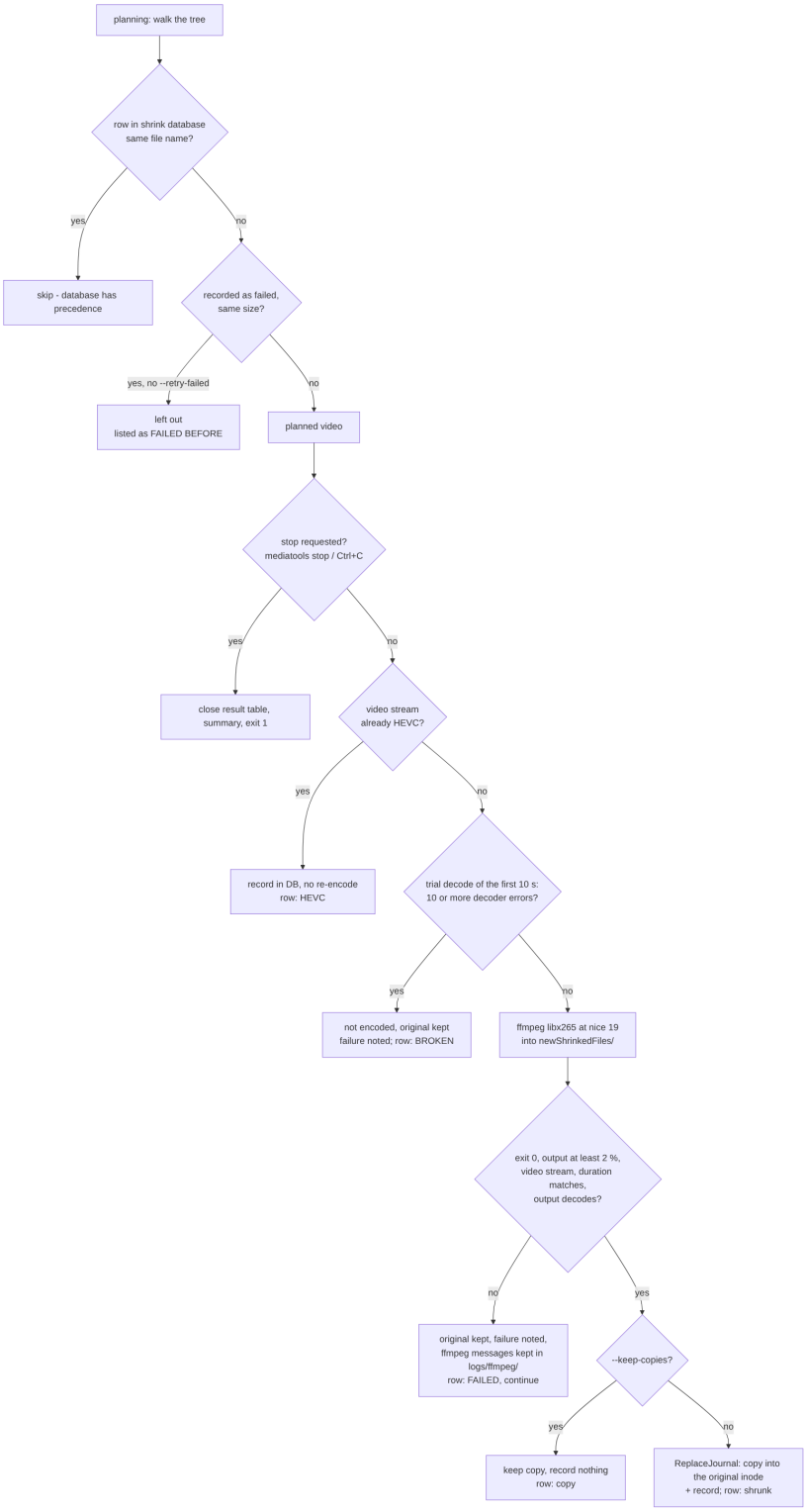


Figure 11: Decision per file

Invariants:

- the database is consulted before anything else;
- copy-back uses `shutil.copyfile`, which writes into the existing inode;
- copy-back and `record()` are not separated by a stop check.

5.2 triage

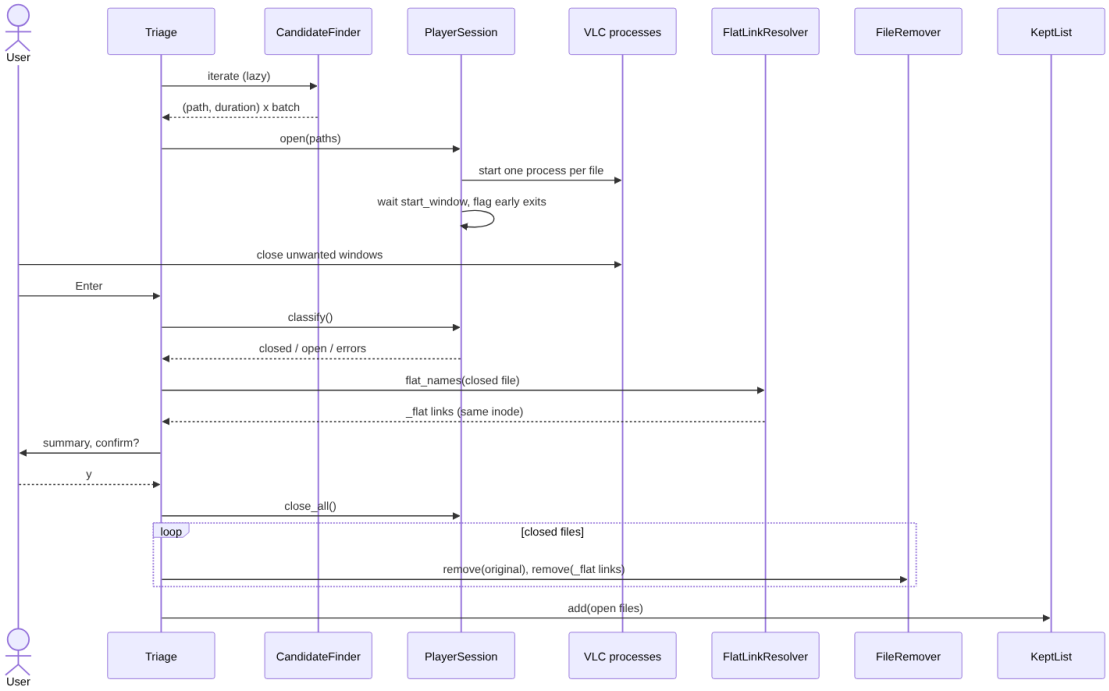


Figure 12: Sequence of one batch

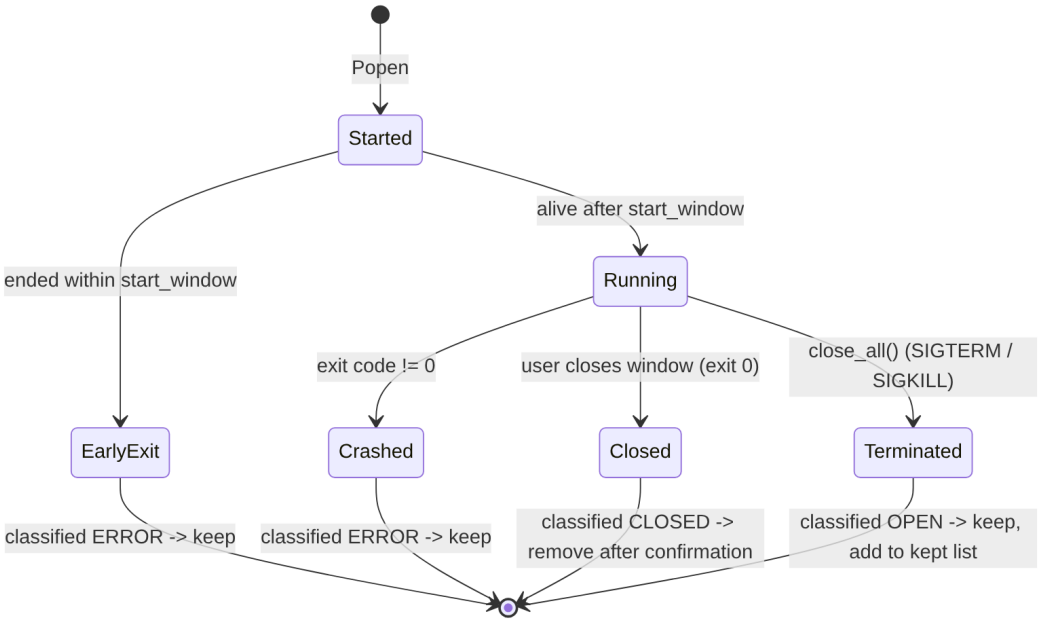


Figure 13: States of a player process

5.3 flatten

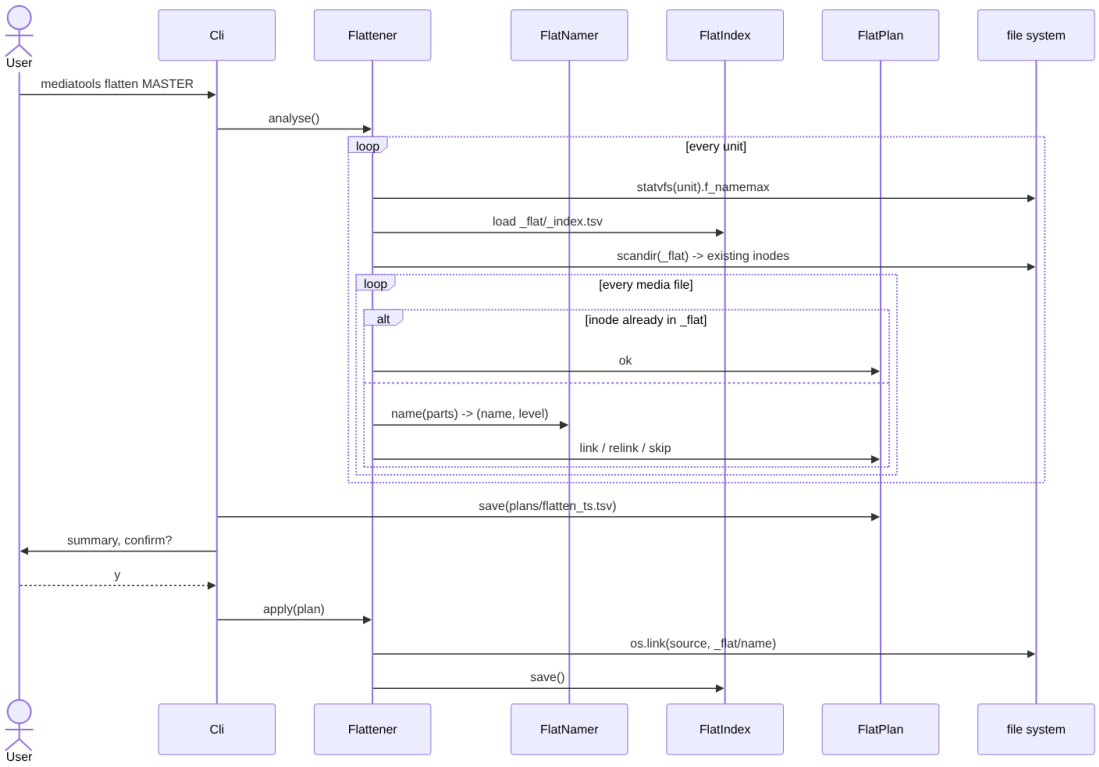


Figure 14: Sequence of a flatten run

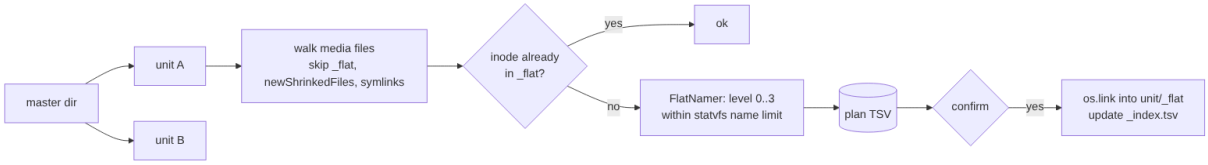


Figure 15: Activity per unit

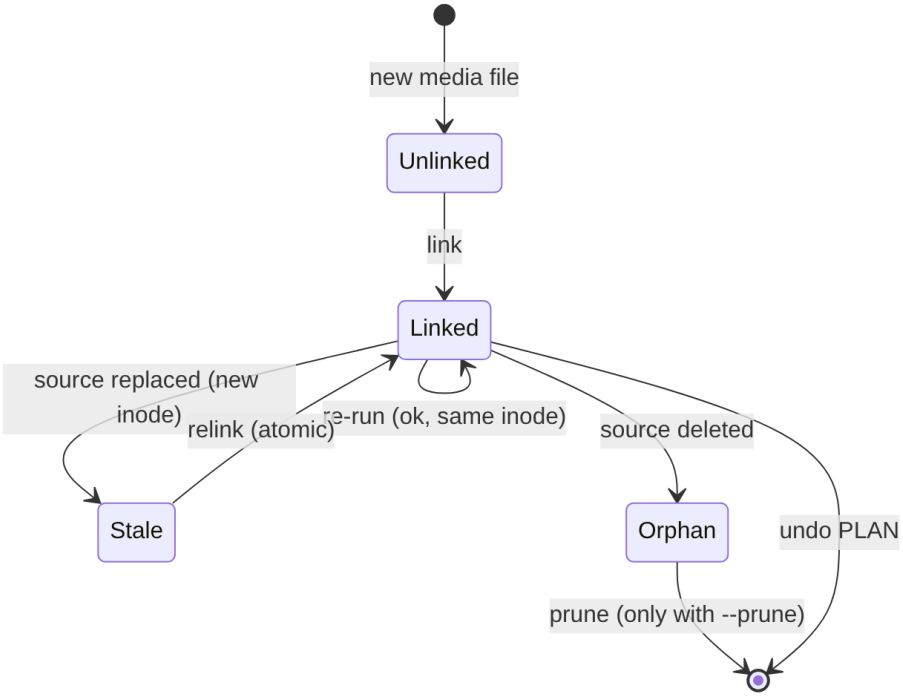


Figure 16: States of a _flat entry

5.4 sync

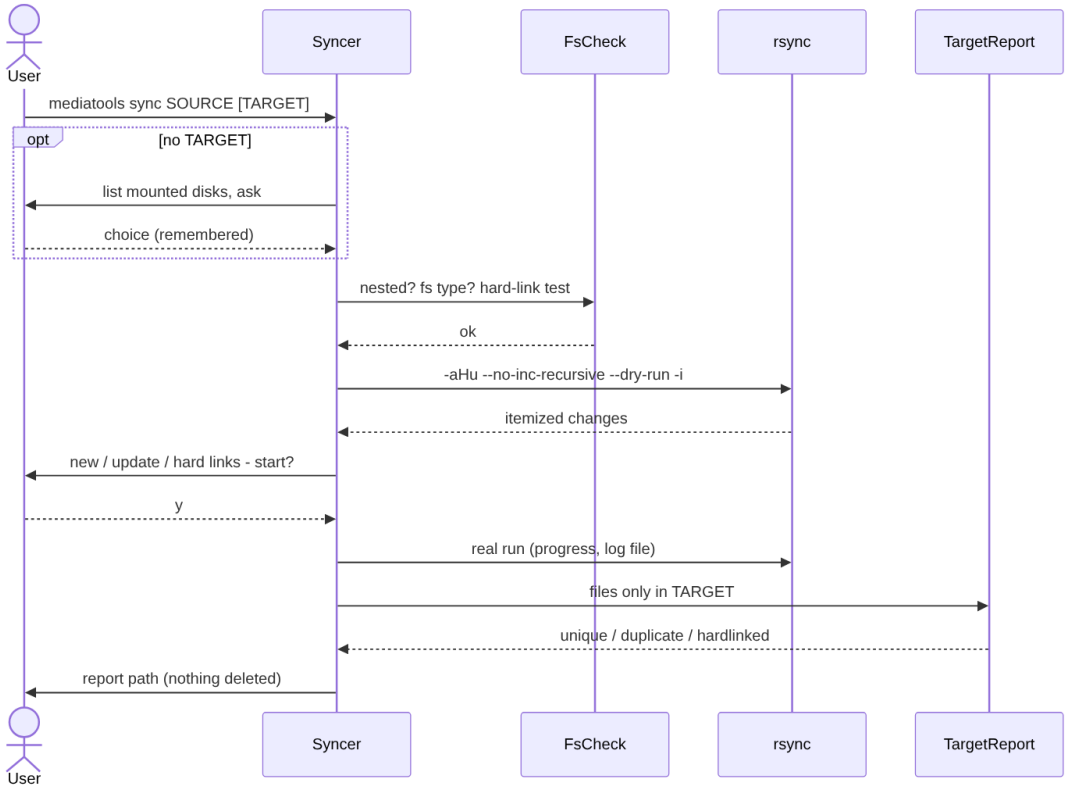


Figure 17: Sequence of a sync run

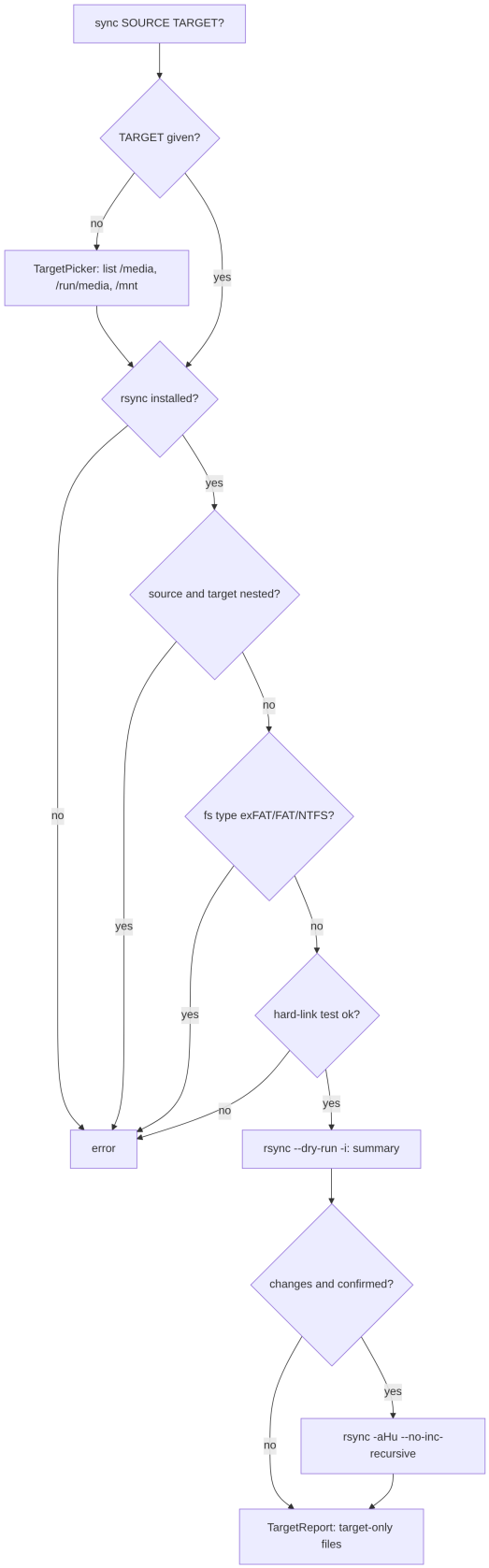


Figure 18: Safety checks

5.5 snapshot

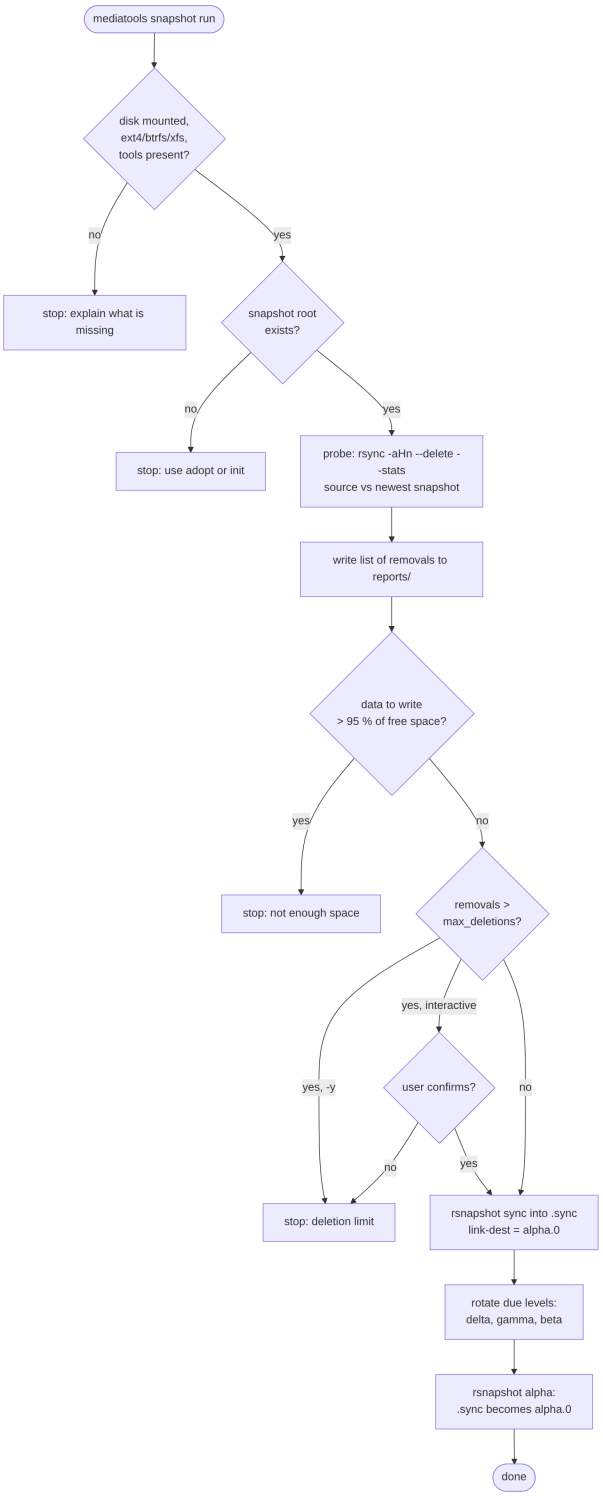


Figure 19: Activity of a snapshot run

Invariants:

- nothing on the disk is changed before a probe has run;
- `adopt` only renames within one file system and only after the probe has shown that at most 25

% of the source volume differs;
- `stand0` is a hard-link copy of `alpha.0`, never a rename (the next sync needs `alpha.0` as link destination);
- a level counts as rotated only if the inode of `<level>.0` changed.

6 Data formats

```
<data>/mediatools.ini                 Config           INI, see the handbook
<dir>/<db_name>                       ShrinkDatabase   SQLite, files(path PK, size, modified),
                                                       failures(name PK, size, status, reason, at)
<dir>/<db_name>.bak_<ts>              ShrinkDatabase   copy before schema migration
<unit>/_flat/_index.tsv               FlatIndex        link name TAB relative source path
<data>/plans/flatten_<ts>.tsv         FlatPlan         action unit link source level note
<data>/state/triage_kept.tsv          KeptList         path TAB size TAB mtime
<data>/state/triage_discarded.tsv     Triage           time TAB size TAB path
<data>/state/triage_durations.tsv     DurationCache    path TAB size TAB mtime_ns TAB seconds
<data>/state/sync_targets.json        TargetPicker     {source: target}
<data>/reports/progress_*_<ts>.txt    ProgressReport   one path per line
<data>/state/shrink_throughput.json   ThroughputLog    {bytes, seconds}
<dir>/newShrinkedFiles/<n>.replace    ReplaceJournal   journal marker during copy-back
<data>/logs/ffmpeg/<ts>_<n>.ffmpeg.log Shrinker         complete ffmpeg messages of a failed encode
<data>/reports/sync_target_only_*.tsv TargetReport     status size path same_as
<data>/logs/<command>_<ts>.log        RunLog           timestamp level message
<data>/snapshot/rsnapshot.conf        RsnapshotConf    generated, tab separated
<data>/state/snapshot_levels.json     LevelSchedule    {level: ISO time}
<data>/reports/snapshot_*_<ts>.txt    Snapshotter      removed paths, one per line
<disk>/<root>/<level>.<n>/...         rsnapshot        snapshot; source at localhost/<path>
<disk>/<root>/stand0/...              Snapshotter      first state, kept permanently
```

All text files are UTF-8; file names that are not valid UTF-8 are written with `surrogateescape`, so they round-trip unchanged.

7 Error handling, signals, exit codes

Situation	Behaviour
missing external tool	`Cli._need` raises `SystemExit` with the tool names
ffmpeg fails / output invalid (size, stream, duration, decode)	file logged as FAILED, failure row, message file kept, original kept, run continues, exit 2
trial decode finds undecodable data	file set aside as BROKEN before encoding, failure row, original kept, run continues, exit code unchanged
video failed in an earlier run, size unchanged	left out in planning unless `--retry-failed`
second `mediatools shrink` while one is alive	refused before planning, exit 3
`mediatools reload` while a shrink runs	switch after the current file (`Shrinker.RESTART` = 4, internal), process replaced by the installed release; a release that does not start: request dropped, run continues
switch cannot exec	message on stderr, exit 1; the next `mediatools shrink` continues with the files left
Ctrl+C in shrink	flag set; current ffmpeg stops; original untouched; exit 1
Ctrl+C in triage	all players closed, nothing removed, exit 130
link creation fails	logged, counted as `error`, flatten exits 1
rsync exit 24 (vanished files)	accepted as success
target cannot hold hard links	sync refuses before copying anything

Situation	Behaviour
configuration value missing or invalid	`SystemExit` naming key, file and expected format
snapshot probe implausible	adopt/run refuse; top-level listings shown
deletion limit exceeded with `-y`	snapshot run aborts, exit 1

8 Testing

8.1 Strategy

- **Unit tests** per class with real files in `tmp_path`.
- **Test doubles** replace interaction: `FakePrompt` (fixed answers), `FakePlayer` (first window "closed"), the `fake_player` fixture (a shell script that behaves like a GUI player: SIGUSR1 = window closed, exit 0).
- **Integration tests** use the real ffmpeg, rsync and rsnapshot; they are skipped via `needs_ffmpeg` / `needs_rsync` / `needs_rsnapshot` when the tool is missing. Snapshot tests replace the mount check (`ismount=`) and the mount table (`FsCheck(mounts_file)`), so they run in a temporary directory.
- **End-to-end tests** call `Cli().main([...])`.
- Every test gets its own `MEDIATOOLS_HOME` (autouse fixture `datadir`).

8.2 Coverage of the safety rules

```
database precedence, HEVC skip, inode kept   test_shrinker_full_cycle
truncated output rejected                     test_encoder_rejects_duration_mismatch
progress is read-only                         test_progress_is_read_only
early player exit never deletes               test_player_early_exit_is_error
_flat link removed with the original          test_triage_removes_original_and_flat_link
no removal without confirmation               test_triage_not_confirmed_removes_nothing
database rows of a moved collection count     test_database_matches_by_name_after_move
1st Ctrl+C finishes, 2nd keeps the original   test_first_ctrl_c_finishes_current_file,
                                              test_second_ctrl_c_aborts_without_touching_original
'mediatools stop' finishes the current file   test_stop_command_finishes_current_file_and_ends
no stop file without a running job            test_no_stop_file_without_a_running_job
interrupted copy-back is completed            test_interrupted_replacement_is_completed
private terms never reach a push              test_finds_terms_in_files_paths_and_messages
placeholder left, never overwrites a file     test_triage_removes_original_and_flat_link,
                                              test_placeholder_never_overwrites
placeholder never replaces a backup copy      test_sync_placeholder_keeps_backup_copy
originals untouched by flatten                test_flatten_never_touches_originals
hard links preserved, nothing deleted         test_sync_preserves_hardlinks_and_never_deletes
newer receiver file kept (rsync -u)           test_sync_update_does_not_overwrite_newer_target
wrong directory never adopted                 test_adopt_refuses_wrong_directory
adopt copies nothing, removals kept in stand0 test_adopt_then_run_keeps_removed_files
deletion limit stops unattended runs          test_adopt_then_run_keeps_removed_files
unmounted disk stops every action             test_check_requires_mounted_disk
database rename never overwrites              test_renamer_renames_and_respects_conflicts
```

9 Quality gates

All gates live in one script, `tools/check.sh`. The pre-push hook and the CI workflow call the same script, so a green local run predicts a green pipeline. Everything runs as normal user; no root privileges are needed. A full run requires ffmpeg, ffprobe, rsync and rsnapshot: without them the integration tests would be skipped and the coverage would drop, so the script stops with an installation hint instead.

```
pip install -e '.[dev]'                # tools: pytest, pytest-cov, pylint,
                                       #        mypy, pycodestyle, bandit
git config core.hooksPath .githooks    # once per clone
tools/check.sh                         # all gates
tools/check.sh --fast                  # gates 1-4 only
```

No.	Gate	Tool	Pass criterion
0	privacy	privacy_scan	no locally listed term in files, paths, commits
1	format	pycodestyle	PEP 8 layout, max line length 100
2	lint	pylint	10.00/10, docparams/docstyle (Google style)
3	types	mypy	no error; `check_untyped_defs`, `warn_unreachable`
4	security	bandit	no finding
5	tests	pytest	all pass, branch coverage >= 90 %

Privacy gate. `tools/privacy_scan.py` searches all tracked files (PDFs via `pdftotext`), file paths and commit messages for terms that would allow conclusions about the processed data. The terms are never in the repository; they come from `$MEDIATOOLS_HOME/privacy_denylist.txt` (one term per line) and from the unit directory names below `[collection] root`. Matching ignores case and whole words only (`cat` does not match `concatenate`); terms shorter than three characters are ignored. Without local terms the gate is skipped, which keeps CI neutral.

Ordinary words can be on the deny list and still be needed in technical text. Instead of rewording, such uses are permitted by **allowed phrases** in `$MEDIATOOLS_HOME/privacy_allowlist.txt` (same format, one phrase per line). They are blanked out before the search: the term stays forbidden everywhere else. Both lists stay outside the repository.

```
"${EDITOR:-nano}" ~/scripts/mediatools.d/privacy_denylist.txt
"${EDITOR:-nano}" ~/scripts/mediatools.d/privacy_allowlist.txt
python3 tools/privacy_scan.py
```

Notes on the configuration (`pyproject.toml`):

- **bandit** skips B404 and B603: the package calls ffmpeg, ffprobe, rsync, the player and gio deliberately via `subprocess` with fixed argument lists and never with `shell=True`. B324 (SHA-1) was fixed with `usedforsecurity=False`; the hash only shortens file names.
- **pytest** turns `ResourceWarning` and unraisable exceptions into errors. This gate found two real leaks during development: SQLite connections that were committed but never closed (`with sqlite3.connect()` does not close), and log handlers of recycled loggers that were dropped without `close()`.
- **coverage** compares with two decimals (`precision = 2`); with the default precision 0, 79.66 % is rounded to 80 % and passes a threshold of 80 % although the report prints “FAIL”.
- **mypy** runs in the moderate mode above; `--strict` is not a gate yet (34 findings, mostly missing generic parameters in tests).
- `python3 tools/gen_docs.py` followed by an unchanged `git status` shows that diagrams and API reference are up to date.

The pre-push hook (`.githooks/pre-push`) runs only when branches are pushed, not for tag-only pushes. `git push --no-verify` bypasses it in an emergency; CI still enforces all gates.

10 Branching model

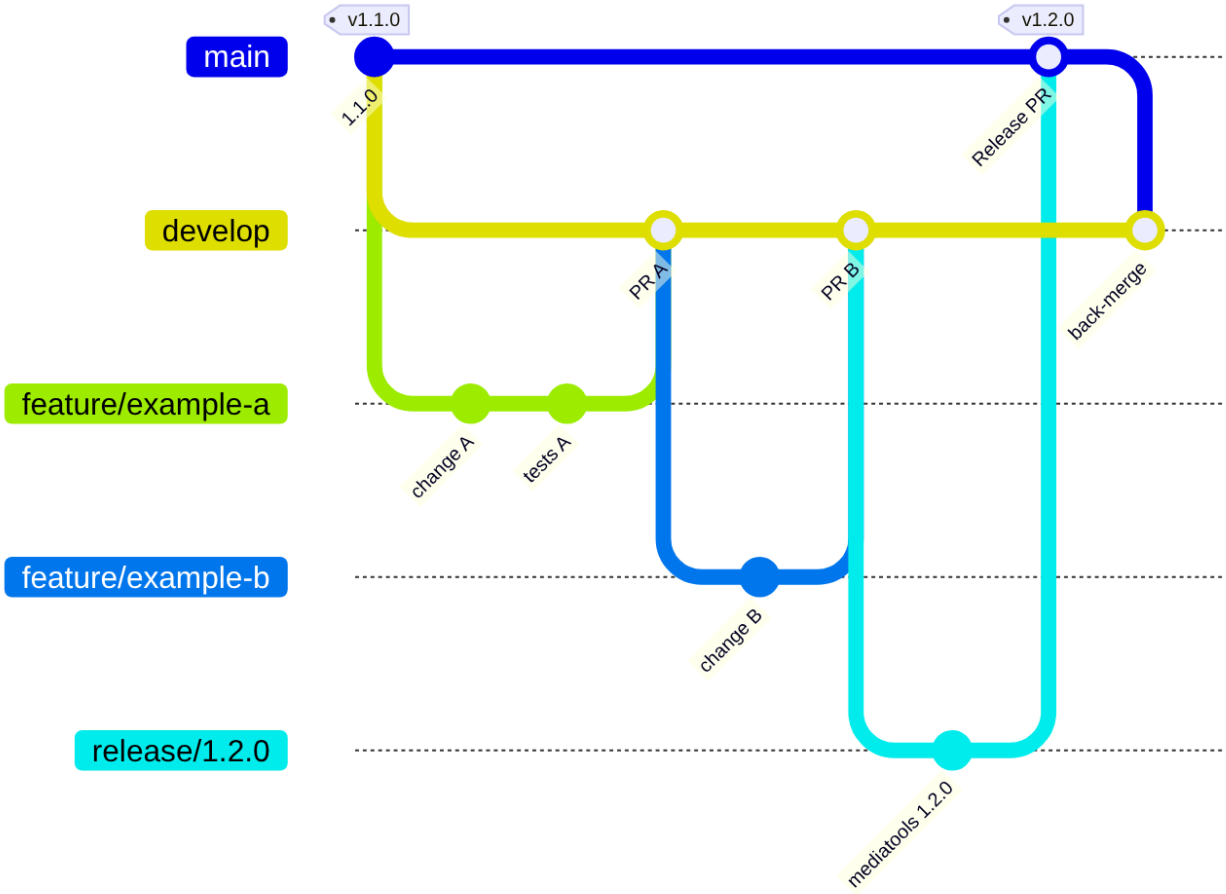


Figure 20: Branching model (illustrative): feature branches merge into develop; a release branch goes into main by pull request, is tagged there and merged back into develop

Branch	Purpose	From	Into
`main`	released states only, each one tagged	-	-
`develop`	integration of finished features	`main`	`main`
`feature/<x>`	one feature or fix	`develop`	`develop` (PR)
`release/<x>`	version, CHANGELOG and PDFs	`develop`	`main` (PR)
`hotfix/<x>`	urgent fix of a release	`main`	`main` + `develop`

Roles: developers work on `feature/*` and open pull requests into `develop`. The **integrator** reviews and merges them, checks `develop` as a whole and decides when a release is cut.

CI (`.github/workflows/tests.yml`) runs the gates on pull requests into `main` and `develop`, on pushes to `main` and `develop` (the merges) and on tags `v*`. Work branches are checked through their pull request only; a push trigger for them as well would run every gate twice. A branch without a pull request can be checked on demand: `gh workflow run quality-gates --ref BRANCH`. For a tag, the extra job `release-check` verifies that the tagged commit lies on `main` and that the tag equals `v + __version__`.

10.1 Branch protection

`main` and `develop` are protected on GitHub: changes only by pull request, merge only with all three `gates` jobs green and the branch up to date, the rule applies to administrators too, no force push, no deletion. Set once per branch by the repository owner (normal user; `gh` acts with the owner's token):

```
for branch in develop main; do
gh api -X PUT "repos/peddanet/mediatools/branches/$branch/protection" --input - <<'EOF'
{
  "required_status_checks": {
    "strict": true,
    "contexts": ["gates (3.10)", "gates (3.12)", "gates (3.13)"]
  },
  "enforce_admins": true,
  "required_pull_request_reviews": { "required_approving_review_count": 0 },
  "restrictions": null,
  "allow_force_pushes": false,
  "allow_deletions": false
}
EOF
done
```

`required_approving_review_count: 0` demands a pull request but no second person's approval (single maintainer). Tags are not affected by branch protection; the integrator pushes release tags directly.

10.2 Feature cycle

```
git switch develop && git pull --ff-only
git switch -c feature/<name>
# ... edit, commit ...
tools/check.sh
git push -u origin feature/<name>        # hook runs the gates again
gh pr create --base develop --fill       # or in the web UI
# after review and green CI: merge the PR, then
git switch develop && git pull --ff-only
git branch -d feature/<name>
git push origin --delete feature/<name>
```

11 Building the documentation

```
sudo apt install pandoc texlive-xetex texlive-latex-extra fonts-dejavu
npm install -g @mermaid-js/mermaid-cli
```

`sudo` is needed because apt and a global npm installation write to system directories. The build itself runs as normal user:

```
cd ~/Projects/mediatools
python3 tools/gen_docs.py
cd docs/img && for f in *.mmd; do mmdc -i "$f" -o "${f%.mmd}.png" -s 2 -b white; done
cd .. && pandoc developer.md api.md -o developer.pdf --pdf-engine=xelatex \
  --highlight-style=tango --resource-path=.
pandoc handbook.md -o handbook.pdf --pdf-engine=xelatex --highlight-style=tango \
  --resource-path=.
```

12 Release procedure

Performed by the integrator. Because `main` and `develop` are protected, every step that changes a branch goes through a pull request.

1. Release branch from an up-to-date `develop`; set `__version__` in `src/mediatools/__init__.py` (Semantic Versioning), turn the `[Unreleased]` section of `CHANGELOG.md` into `[X.Y.Z] - date`, rebuild diagrams and PDFs, run `tools/check.sh`, commit "mediatools X.Y.Z":

```
git switch develop && git pull --ff-only
git switch -c release/X.Y.Z
# ... version, CHANGELOG, PDFs ...
git commit -am "mediatools X.Y.Z"
git push -u origin release/X.Y.Z
gh pr create --base main --title "Release X.Y.Z" --fill
gh pr checks --watch && gh pr merge --merge --delete-branch
```

2. Tag the merge commit on `main` and push the tag; CI runs `release-check` for it:

```
git switch main && git pull --ff-only
git tag -a vX.Y.Z -m "mediatools X.Y.Z"
git push origin vX.Y.Z
```

3. Bring the release back into `develop` (pull request from `main`):

```
gh pr create --base develop --head main --title "Merge release X.Y.Z into develop" \
  --body "Back-merge of release X.Y.Z."
gh pr checks --watch && gh pr merge --merge
git switch develop && git pull --ff-only
```

4. Install the release on the own machine: `./install.sh` (newest tag).

Merge commits (no squash, no rebase) keep every commit; `git log --first-parent main` lists exactly the releases. A hotfix branches from `main`, goes into `main` by pull request (tag `vX.Y.Z+1`) and back into `develop` as in step 3.

13 Applying delivered patch archives

Hotfixes prepared outside the clone arrive as archives `mediatools-hotfix-X.Y.Z-patch.tar.gz` (output of `git format-patch`, one release per archive). `mediatools update` applies them. It runs the shell script `src/mediatools/update/apply-release.sh`, which is shipped inside the package (package data), so every installed release carries it; the same script serves every release:

```
mediatools update --list     # what is there, nothing changes
mediatools update            # asks, then applies
mediatools update -y         # all pending versions, no question
```

Step	What happens
inventory	archives in `~/Downloads` (`--downloads DIR`); per version the most recently saved copy (browser copies `... (1).tar.gz`)
classification	versions above the release on `main` are *pending*, the rest and older copies are *stale*
choice	a version means "all pending versions up to it, in ascending order", because each patch builds on the release before it; Enter = all
per version	archive check (`__version__` in the patch) -> `hotfix/X` from `main` -> `git am` -> push (pre-push gates) -> PR into `main` -> CI -> merge -> tag `vX` -> CI of the tag -> back-merge into `develop` via `merge/hotfix-X`
install	`install.sh` (newest tag), then `mediatools --version` must name the last applied version

Step	What happens
tidying	used archives to `~/scripts/mediatools.d/releases/` (plain names), stale archives and old `apply-mediatools-*.sh` scripts deleted

The script copies itself to `/tmp` and runs from there, because switching branches and the installation rewrite the script while bash is still reading it. It finds the clone next to itself (source checkout) or uses `~/Projects/mediatools` (`--repo DIR`). It logs to `~/scripts/mediatools.d/logs/apply-release_<ts>.log` and stops at the first failing command; nothing in the download folder is removed then, and the same command can be repeated (hotfix branch rebuilt, open pull request reused). It runs as a normal user; root would leave root-owned files in the clone.

14 Adding a subcommand

1. Create `src/mediatools/<feature>/` with one class per module and an `__init__.py` that exports the classes.
2. Import only from `mediatools.core`.
3. Add a sub-parser in `Cli._build_parser` and a method `cmd_<name>` that wires the objects and returns an exit code.
4. Add tests (unit, integration, CLI).
5. Run the quality gates; regenerate diagrams and API reference.
6. Document the command in `README.md`, `docs/handbook.md` and here.

15 Glossary

Bandit – static security analyser for Python (PyCQA). bandit configuration

Activity diagram – UML diagram of a decision flow (here Mermaid flowcharts). UML specification (OMG)

Dependency injection – collaborators are passed in instead of being created inside a class.

docparams – pylint extension that checks parameter, return, yield and raise documentation. pylint docparams

git-flow – branching model with the long-lived branches `main` (releases) and `develop` (integration) plus short-lived feature, release and hotfix branches (V. Driessen, 2010). A successful Git branching model

Google-style docstring – docstring layout with `Args:`, `Returns:`, `Raises:` sections. Google Python Style Guide

Integrator – role that reviews and merges pull requests into `develop` and cuts releases (merge into `main`, tag).

mypy – static type checker for Python.

pre-push hook – script in the hooks directory (`core.hooksPath`) that git runs before a push; a non-zero exit code aborts the push.

Branch protection – GitHub rule that blocks direct pushes, force pushes and merges without passing status checks on a branch. About protected branches

Mermaid – text format for diagrams, rendered by GitHub and `mmdc`. mermaid.js.org

Sequence diagram – UML diagram of messages between participants over time.

State diagram – UML diagram of the states of one object and the transitions between them.

Test double – replacement object used in tests instead of a real collaborator.

16 Sources

Source	Status
pylint extensions (docparams)	standard reference; behaviour checked with pylint 4.1.3
Google Python Style Guide, section 3.8 Comments and Docstrings	standard reference, not retrieved in this session
Mermaid documentation	standard reference; diagrams rendered with mermaid-cli
UML specification (OMG)	standard reference, not retrieved in this session
V. Driessen: A successful Git branching model (2010)	found via search 2026-11-03; original domain not fetchable from the build sandbox, content confirmed through mirrors
GitHub Docs: About protected branches	found via search 2026-11-03; required status checks and PR requirement confirmed
bandit configuration (1.9.3)	found via search 2026-11-03; `[tool.bandit]` requires `-c pyproject.toml`
Python docs: sqlite3, connection context manager	not fetchable from the sandbox; note “neither … opens a new transaction nor closes the connection” confirmed via the quote in discuss.python.org #41663
mypy configuration file	standard reference, not retrieved; behaviour checked with the installed mypy
git: githooks(5)	standard reference, not retrieved; pre-push stdin format checked against local git
User handbook `docs/handbook.md`	sources for rsync, eCryptfs, ffmpeg, SQLite

17 Appendix: API reference

Generated by `tools/gen_docs.py` from the docstrings (Google style). Do not edit by hand.

17.1 `mediatools.cli`

`Cli` (`mediatools.cli`)

```
Parses arguments and dispatches to the subcommand classes.

Each ``cmd_*`` method builds the objects of one subcommand, runs it and
returns the process exit code.

Attributes:
    parser: The configured :class:`argparse.ArgumentParser`.
```

`__init__`

```
__init__(self)

Build the argument parser.
```

`_add_shrink_parsers`

```
_add_shrink_parsers(sub: argparse._SubParsersAction) -> None

Define ``shrink`` and the commands around a running shrink.

``stop`` and ``attach`` act on a shrink run in the background;
``update`` installs a new release.

Args:
    sub: Sub-parser collection of the main parser.
```

`_build_parser`

```
_build_parser() -> argparse.ArgumentParser

Define all subcommands and options.

Returns:
    The parser.
```

`_add_admin_parsers`

```
_add_admin_parsers(sub) -> None

Define the subcommands rename-db, config and snapshot.

Args:
    sub: Sub-parser collection of the main parser.
```

`main`

```
main(self, argv: list[str] | None = None) -> int

Parse arguments and run the selected subcommand.

Args:
    argv: Arguments without the program name; ``sys.argv`` if None.

Returns:
    Process exit code.
```

`_dir`

```
_dir(path: str) -> str

Validate a directory argument.

Args:
    path: Directory given on the command line.

Returns:
    Absolute path.

Raises:
    SystemExit: If the path is not a directory.
```

`_need`

```
_need(*tools: str) -> None

Ensure external programs are installed.

Args:
    *tools: Executable names.

Raises:
    SystemExit: If at least one tool is missing.
```

`cmd_shrink`

```
cmd_shrink(self, args: argparse.Namespace) -> int

Run ``mediatools shrink``.

Args:
    args: Parsed arguments.

Returns:
    Exit code of :meth:`Shrinker.run` (0 if the user declined), or 3
    if another shrink run is still active.
```

`_switch_release`

```
_switch_release(restart: SelfRestart, run: RunLog) -> int

Replace this process by the installed release (see :class:`SelfRestart`).

Args:
    restart: Command line and log of the new process.
    run: Log of this run; closed before the switch.

Returns:
    1 - only if the new process could not be started.
```

`cmd_reload`

```
cmd_reload(_args: argparse.Namespace) -> int

Run ``mediatools reload``: ask the running shrink to switch releases.

Returns:
    0 if a running shrink was asked, 1 if none runs.
```

`cmd_stop`

```
cmd_stop(_args: argparse.Namespace) -> int

Run ``mediatools stop``: request a stop of the running shrink.

Returns:
    0 if a running shrink was asked to stop, 1 if none runs.
```

cmd_attach

```
cmd_attach(_args: argparse.Namespace) -> int

Run ``mediatools attach`` (alias ``front``): watch a background shrink.

Returns:
    0 after showing a run, 1 if none runs.
```

cmd_front

```
cmd_front(_args: argparse.Namespace) -> int

Run ``mediatools attach`` (alias ``front``): watch a background shrink.

Returns:
    0 after showing a run, 1 if none runs.
```

cmd_update

```
cmd_update(args: argparse.Namespace) -> int

Run ``mediatools update``: apply delivered release patches.

Args:
    args: Parsed arguments.

Returns:
    Exit code of the bundled release script.
```

cmd_progress

```
cmd_progress(self, args: argparse.Namespace) -> int

Run ``mediatools progress``.

Args:
    args: Parsed arguments.

Returns:
    Exit code of :meth:`ProgressReport.run`.
```

cmd_triage

```
cmd_triage(self, args: argparse.Namespace) -> int

Run ``mediatools triage``.

Args:
    args: Parsed arguments.

Returns:
    Exit code of :meth:`Triage.run` (0 for a dry run).

Raises:
    SystemExit: If the duration range is invalid.
```

cmd_flatten

```
cmd_flatten(self, args: argparse.Namespace) -> int

Run ``mediatools flatten`` (analysis, plan, confirmation, apply).

Args:
    args: Parsed arguments.

Returns:
    0 on success or dry run, 1 if a link could not be created.

Raises:
    SystemExit: If no path is given and none is configured.
```

_log_plan

```
_log_plan(logger, plan: FlatPlan, plan_path: str) -> int

Log the summary of a flatten plan.

Args:
    logger: :class:`logging.Logger`.
    plan: Plan from :meth:`Flattener.analyse`.
    plan_path: Where the plan was saved.

Returns:
    Number of changes the plan would make.
```

cmd_sync

```
cmd_sync(self, args: argparse.Namespace) -> int

Run ``mediatools sync``.

Args:
    args: Parsed arguments.

Returns:
    Exit code of :meth:`Syncer.run`; 1 if no target was chosen.
```

cmd_rename_db

```
cmd_rename_db(self, args: argparse.Namespace) -> int

Run ``mediatools rename-db``.

Args:
    args: Parsed arguments.

Returns:
    0 on success or dry run, 1 if a file could not be renamed.
```

cmd_config

```
cmd_config(args: argparse.Namespace) -> int

Run ``mediatools config``.

Args:
    args: Parsed arguments.

Returns:
    0 on success.

Raises:
    SystemExit: If a ``--set`` item is not ``SECTION.KEY=VALUE``.
```

cmd_snapshot

```
cmd_snapshot(self, args: argparse.Namespace) -> int

Run ``mediatools snapshot <action>``.

Args:
    args: Parsed arguments.

Returns:
    Exit code of the action.

Raises:
    SystemExit: If ``adopt`` is called without a directory.
```

`_install_trigger`

```
_install_trigger(settings: SnapshotSettings, logger) -> int

Install the systemd user unit after checking the mount unit.

Args:
    settings: Validated snapshot settings.
    logger: :class:`logging.Logger`.

Returns:
    0 on success, 1 if the disk's mount unit is not known.
```

17.2 `mediatools.core`

`AttachView (mediatools.core.attachview)`

```
Follow the log and the progress lines of a running job.

Shows the last :attr:`BACKLOG` lines of the job's log file, then every
new line as it is written - like the terminal of the run: DEBUG lines
are left out, INFO lines without time stamp and level - and below them
the two progress lines from the status file, redrawn in place. Ctrl+C
only leaves the view; the job keeps running. The view ends by itself
when the job ends.

Example:
    >>> AttachView(RunControl("shrink")).run()

Attributes:
    control: Control files of the job.
    stream: Output stream (the terminal).
    poll: Seconds between two updates.
    drawn: True while the progress lines are on the screen.
```

`__init__`

```
__init__(
    self,
    control: RunControl,
    stream: TextIO | None = None,
    poll: float = 1.0,
) -> None

Configure the view.

Args:
    control: Control files of the job.
    stream: Output stream; stdout if None.
    poll: Seconds between two updates.
```

`run`

```
run(self) -> int

Show the job until it ends or the user presses Ctrl+C.

Returns:
    0 after showing a job, 1 if none runs.
```

_backlog

```
_backlog(self, log: str | None) -> int

Print the last lines of the log.

Args:
    log: Log file, or None.

Returns:
    Offset after the printed part.
```

_follow

```
_follow(self, log: str | None, offset: int) -> int

Print log lines written since ``offset``.

Args:
    log: Log file, or None.
    offset: Position after the last printed line.

Returns:
    New position.
```

terminal_lines

```
terminal_lines(cls, text: str) -> list[str]

Turn log file lines into what the terminal of the run shows.

Args:
    text: Complete log lines.

Returns:
    Lines with line break: DEBUG dropped, INFO without prefix,
    other levels as ``LEVEL message``; foreign lines unchanged.
```

_draw

```
_draw(self) -> None

Redraw the two progress lines from the status file.
```

_clear

```
_clear(self) -> None

Remove the progress lines before new log lines are printed.
```

CodecCache (mediatools.core.codeccache)

```
Remembers the video codec of files keyed by path, size and mtime.

The shrink plan asks for the codec of every candidate, so that videos
which are already HEVC can be reported (and left out of volume and time
estimates) before the start question. Probing costs one ffprobe process
per file; the cache answers repeated plans almost instantly. An entry is
used only while size and modification time are unchanged, so a file
that was shrunk in place is probed again. Storage and lookup are those
of :class:`DurationCache`; only the value is a codec name.

Stored as TSV ``path<TAB>size<TAB>mtime_ns<TAB>codec``.
```

parse

```
parse(text: str) -> Any

Convert a stored value.

Args:
    text: Value column of the TSV file.

Returns:
    Codec name.
```

format

```
format(value: Any) -> str

Convert a value for storage.

Args:
    value: Codec name.

Returns:
    The codec name.
```

Config (mediatools.core.config)

```
Machine-specific settings kept outside the repository.

Everything that describes a concrete collection - its location, the
name of the shrink databases, folder names worth abbreviating, the
backup disk - lives in this file, so the code and its history allow no
conclusions about the data. The file is plain INI (``configparser``);
a commented template is written by ``mediatools config --init``.

Attributes:
    path: Absolute path of the INI file.
    parser: The loaded :class:`configparser.ConfigParser`.
```

__init__

```
__init__(self, datadir: DataDir | None = None)

Load the configuration; a missing file yields the defaults.

Args:
    datadir: Data directory; the default one if None.
```

exists

```
exists(self) -> bool

Tell whether the INI file exists.

Returns:
    True if the file exists.
```

write_template

```
write_template(self) -> bool

Write the commented template unless the file already exists.

Returns:
    True if the file was written, False if it already existed.
```

set_value

```
set_value(self, section: str, key: str, value: str) -> None

Set one value in the INI file, keeping comments and layout.

The file is created from the template if it does not exist. Only
the line ``key = ...`` inside the section is replaced (or added at
the end of the section).

Args:
    section: INI section.
    key: Key inside the section.
    value: New value.
```

get

```
get(self, section: str, key: str) -> str

Return a raw value.

Args:
    section: INI section.
    key: Key inside the section.

Returns:
    Stripped value; empty string if unset.
```

path_value

```
path_value(self, section: str, key: str) -> str | None

Return a path value with ``~`` expanded.

Args:
    section: INI section.
    key: Key inside the section.

Returns:
    Absolute path, or None if the value is empty.
```

integer

```
integer(self, section: str, key: str) -> int

Return an integer value.

Args:
    section: INI section.
    key: Key inside the section.

Returns:
    The value.

Raises:
    SystemExit: If the value is not an integer.
```

db_name

```
db_name(self) -> str

Return the file name of the shrink databases.

Returns:
    File name without directory.

Raises:
    SystemExit: If the value contains a path separator.
```

abbreviations

```
abbreviations(self) -> dict[str, str]

Return the folder abbreviations of naming level 1.

Returns:
    Mapping folder name to abbreviation.

Raises:
    SystemExit: If an entry is not of the form ``Name=Abbreviation``.
```

require_path

```
require_path(self, section: str, key: str, given: str | None) -> str

Return a command-line path or, if omitted, the configured one.

Args:
    section: INI section of the fallback value.
    key: Key of the fallback value.
    given: Path from the command line or None.

Returns:
    Path to use (not yet validated).

Raises:
    SystemExit: If neither is available; the message explains how
        to configure the value.
```

render

```
render(self) -> str

Return the effective configuration as INI text.

Returns:
    All sections and values, defaults included.
```

DataDir (mediatools.core.datadir)

```
Runtime data directory of mediatools.

The root is taken from the environment variable ``MEDIATOOLS_HOME``;
the default is ``~/scripts/mediatools.d``. Nothing is written into the
media collection itself except what a subcommand documents (shrink
databases, ``_flat`` directories).

Layout::

    <root>/logs/      one log file per run, date and time in the name
    <root>/state/     persistent state, e.g. triage kept list
    <root>/plans/     flatten plans (for review and undo)
    <root>/reports/   progress lists and sync reports

Attributes:
    root: Absolute path of the data root.
```

__init__

```
__init__(self, root: str | None = None)

Resolve the data root.

Args:
    root: Explicit root; if None, ``$MEDIATOOLS_HOME`` or the default
        is used.
```

sub

```
sub(self, name: str) -> str

Return a subdirectory of the data root, creating it if needed.

Args:
    name: Subdirectory name, e.g. ``logs``.

Returns:
    Absolute path of the subdirectory.
```

file

```
file(self, sub: str, prefix: str, suffix: str) -> str

Build a time-stamped file path inside a subdirectory.

Args:
    sub: Subdirectory name, e.g. ``logs``.
    prefix: File name prefix, e.g. ``shrink``.
    suffix: File name suffix including the dot, e.g. ``.log``.

Returns:
    Path such as ``<root>/logs/shrink_2026-11-03_13-12-52.log``.
```

stamp

```
stamp() -> str

Return the current time in the format used in file names.

Returns:
    Timestamp ``YYYY-MM-DD_hh-mm-ss``.
```

DecodeCheck (mediatools.core.decodecheck)

```
Decodes the start of a video and counts the decoder errors.

Some videos have an intact container (``ftyp``/``mdat``/``moov``,
plausible duration, codec ``avc1``) but sample data a decoder cannot
use: still encrypted after the encryption boxes were stripped, or
scrambled. Encoding such a file takes as long as a good one and yields a
grey or empty result. A short trial decode finds them in seconds: a
healthy video decodes without errors, an unusable one produces errors
from the first frame on (``decode_slice_header error``, ``no frame!``,
``non-existing PPS``, AAC ``channel element ... is not allocated``).

:meth:`verify` decodes a whole file and accepts it only without a
single error; ``shrink`` uses it on every encoded output before the
original is replaced.

The check reads only; it never writes or changes the file.

Attributes:
    ffmpeg: Name or path of the ffmpeg executable.
    seconds: Length of the trial decode from the start, in seconds.
    max_errors: A file with at least this many error lines is unusable.
    timeout: Seconds after which the trial is abandoned (result unknown).
```

__init__

```
__init__(
    self,
    ffmpeg: str = 'ffmpeg',
    seconds: float = 10.0,
    max_errors: int = 10,
    timeout: float = 300.0,
) -> None

Store the settings.

Args:
    ffmpeg: Name or path of the ffmpeg executable.
    seconds: Length of the trial decode, in seconds.
    max_errors: Error lines from which a file counts as unusable.
    timeout: Seconds after which the trial is abandoned.

Raises:
    ValueError: If ``seconds`` or ``timeout`` is not positive or
        ``max_errors`` is below 1.
```

command

```
command(self, path: str) -> list[str]

Build the ffmpeg command of the trial decode.

All streams are decoded (``-f null``: nothing is written); ``-t``
before ``-i`` limits the input to the first :attr:`seconds`.

Args:
    path: Video file.

Returns:
    Argument list for :func:`subprocess.run`.
```

verify

```
verify(self, path: str, timeout: float, niceness: int = 19) -> str | None

Decode a whole file and report the first problem.

Unlike :meth:`unusable`, an unknown result is a failure here: the
caller keeps the original unless the file decoded completely and
without any error line.

Args:
    path: Video file.
    timeout: Seconds after which the decode is abandoned.
    niceness: Scheduling priority (0 normal, 19 lowest), as for the
        encoder.

Returns:
    None if every stream decoded without error, otherwise the reason.
```

errors

```
errors(self, path: str) -> int | None

Count the error lines of a trial decode.

Args:
    path: Video file.

Returns:
    Number of non-empty lines ffmpeg wrote at log level ``error``,
    or None if ffmpeg is missing or the trial timed out.
```

unusable

```
unusable(self, path: str) -> int | None

Tell whether a video's data cannot be decoded.

Args:
    path: Video file.

Returns:
    The error count if it reaches :attr:`max_errors` (the file is
    unusable), otherwise None - also when the trial could not run,
    so an unknown result never blocks a file.
```

DurationCache (mediatools.core.durationcache)

```
Remembers ffprobe durations keyed by path, size and mtime.

Probing every video costs one ffprobe process. The cache makes the
candidate count of a second run almost instant: an entry is used only
while size and modification time of the file are unchanged.

Stored as TSV ``path<TAB>size<TAB>mtime_ns<TAB>duration``.

Attributes:
    path: Location of the TSV file.
    entries: Mapping path to ``(size, mtime_ns, duration)``.
    hits: Number of lookups answered from the cache in this run.
    added: Number of values stored since the last :meth:`save`.
```

__init__

```
__init__(self, path: str)

Load the cache file if it exists.

Args:
    path: Location of the TSV file (created by :meth:`save`).
```

get

```
get(self, file_path: str, st: os.stat_result) -> Any

Return the cached duration if the file is unchanged.

Args:
    file_path: Video file.
    st: Its current ``os.stat`` result.

Returns:
    Duration in seconds, or None if unknown or outdated.
```

put

```
put(self, file_path: str, st: os.stat_result, duration: Any) -> None

Store a duration.

Args:
    file_path: Video file.
    st: Its ``os.stat`` result at probing time.
    duration: Duration in seconds.
```

save

```
save(self) -> None

Write the cache atomically (temporary file, then rename).
```

parse

```
parse(text: str) -> Any

Convert a stored value.

Args:
    text: Value column of the TSV file.

Returns:
    Duration in seconds.
```

format

```
format(value: Any) -> str

Convert a value for storage.

Args:
    value: Duration in seconds.

Returns:
    Text with millisecond precision.
```

MediaTypes (mediatools.core.mediatypes)

```
Static knowledge about media file extensions and reserved directories.

All subcommands use this class, so "what is a video" and "which
directories are never entered" is defined in exactly one place.

Attributes:
    FLAT_DIR: Name of the per-unit directory created by ``flatten``.
    SHRINK_TMP_DIR: Temporary output directory of ``shrink``.
    SKIP_DIRS: Directory names no subcommand descends into.
    VIDEO: Lower-case video extensions including the dot.
    IMAGE: Lower-case image extensions including the dot.
    AUDIO: Lower-case audio extensions including the dot.
    MEDIA: Union of video, image and audio extensions.
```

extension

```
extension(name: str) -> str

Return the lower-case extension of a file name.

Args:
    name: File name or path.

Returns:
    Extension including the dot, e.g. ``.mp4``; empty if none.
```

is_media

```
is_media(cls, name: str) -> bool

Tell whether a file is a video, image or audio file.

Args:
    name: File name or path.

Returns:
    True if the extension belongs to :attr:`MEDIA`.
```

is_video

```
is_video(cls, name: str) -> bool

Tell whether a file is a video file.

Args:
    name: File name or path.

Returns:
    True if the extension belongs to :attr:`VIDEO`.
```

parse_extensions

```
parse_extensions(cls, csv: str) -> frozenset

Turn a comma separated list into a set of normalised extensions.

Args:
    csv: List such as ``"mp4,MOV, .mkv"``.

Returns:
    Set such as ``{".mp4", ".mov", ".mkv"}``; empty items are ignored.
```

MediaProbe (mediatools.core.probe)

```
Reads technical properties of media files with ``ffprobe``.

Every method returns ``None`` (or False) instead of raising when ffprobe
cannot read a file, so callers decide how to treat unreadable media.

Attributes:
    ffprobe: Name or path of the ffprobe executable.
```

__init__

```
__init__(self, ffprobe: str = 'ffprobe')

Store the executable name.

Args:
    ffprobe: Name or path of the ffprobe executable.
```

available

```
available(self) -> bool

Tell whether ffprobe can be found.

Returns:
    True if the executable is found in ``PATH``.
```

value

```
value(self, path: str, entry: str, stream: str | None = None) -> str | None

Return a single ffprobe value.

Args:
    path: Media file.
    entry: ffprobe ``-show_entries`` argument, e.g. ``format=duration``.
    stream: Optional stream selector, e.g. ``v:0``.

Returns:
    First value printed by ffprobe, or None if ffprobe failed or
    printed nothing.
```

duration

```
duration(self, path: str) -> float | None

Return the container duration.

Args:
    path: Media file.

Returns:
    Duration in seconds, or None if it cannot be determined.
```

video_codec

```
video_codec(self, path: str) -> str | None

Return the codec of the first video stream.

Args:
    path: Media file.

Returns:
    Codec name such as ``h264`` or ``hevc``, or None without video.
```

has_video

```
has_video(self, path: str) -> bool

Tell whether a decodable video stream exists.

Args:
    path: Media file.

Returns:
    True if ffprobe reports a codec for stream ``v:0``.
```

ProgressBar (mediatools.core.progressbar)

```
Shows overall progress by bytes, files and estimated remaining time.

Progress is measured in source bytes, because encoding time grows with
file size far more than with file count. The remaining time is
``remaining bytes / speed``; the speed comes from this run once two
files are done or 30 s have passed, before that from earlier runs
(``initial_rate``).

Two lines are drawn: the overall progress (:meth:`line`) and the
current file (:meth:`file_line`: its percentage, size, encoded part and
remaining time). On a terminal both are redrawn in place and cut to the
terminal width, so they never wrap; otherwise (output redirected to a
file) nothing is drawn and only the log is written.

Attributes:
    total_bytes: Bytes of all files of the run.
    total_files: Number of files of the run.
    done_bytes: Bytes of finished files.
    done_files: Number of finished files.
    initial_rate: Bytes per second from earlier runs, or None.
    stream: Output stream.
    started: Start time (monotonic seconds).
    enabled: True if the stream is a terminal.
    note: Text shown at the start of the first line, e.g. a pending stop.
    drawn: True while the two lines are on the screen.
    status_path: If set, both lines are also written to this file (at
        most once per :attr:`STATUS_SECONDS`), for ``mediatools attach``.
    last_status: Monotonic time of the last status file update.
```

__init__

```
__init__(
    self,
    total_bytes: int,
    total_files: int,
    initial_rate: float | None = None,
    stream: TextIO | None = None,
)

Start the display.

Args:
    total_bytes: Bytes of all files of the run.
    total_files: Number of files of the run.
    initial_rate: Bytes per second from earlier runs, or None.
    stream: Output stream; ``sys.stderr`` if None.
```

rate

```
rate(self, current_bytes: float = 0.0) -> float | None

Return the speed used for the estimate.

Args:
    current_bytes: Bytes of the current file already encoded.

Returns:
    Bytes per second, or None if unknown.
```

remaining_seconds

```
remaining_seconds(self, current_bytes: float = 0.0) -> float | None

Estimate the remaining time.

Args:
    current_bytes: Bytes of the current file already encoded.

Returns:
    Seconds, or None if no speed is known.
```

line

```
line(self, _name: str = '', file_fraction: float = 0.0, file_bytes: int = 0) -> str

Build the line of the overall progress.

The name of the current file is not part of this line (see
:meth:`file_line`); the parameter keeps both methods interchangeable.

Args:
    file_fraction: Encoded fraction of the current file (0..1).
    file_bytes: Size of the current file.

Returns:
    Text without line break.
```

file_line

```
file_line(
    self,
    name: str = '',
    file_fraction: float = 0.0,
    file_bytes: int = 0,
    width: int | None = None,
) -> str

Build the line of the current file.

Args:
    name: Name of the current file.
    file_fraction: Encoded fraction of the current file (0..1).
    file_bytes: Size of the current file.
    width: Maximum length; a longer name is shortened in the middle,
        so that its start and end stay visible. None: no limit.

Returns:
    Text without line break, e.g.
    ``file  35 % of 2.40 GiB (0.84 GiB encoded) | 12m left | clip.mp4``.
```

_write_status

```
_write_status(self, name: str, file_fraction: float, file_bytes: int) -> None

Replace the status file with both lines (rate-limited, atomic).

Args:
    name: Name of the current file.
    file_fraction: Encoded fraction of the current file (0..1).
    file_bytes: Size of the current file.
```

show

```
show(self, name: str = '', file_fraction: float = 0.0, file_bytes: int = 0) -> None

Redraw both status lines on a terminal.

Args:
    name: Name of the current file.
    file_fraction: Encoded fraction of the current file (0..1).
    file_bytes: Size of the current file.
```

file_done

```
file_done(self, size: int) -> None

Count a finished (or skipped) file.

Args:
    size: Its source size in bytes.
```

file_skipped

```
file_skipped(self, size: int) -> None

Remove a file that needed no encoding from the totals.

Such files (already HEVC, recorded meanwhile) cost no encoding time;
counting them as done would inflate progress and speed.

Args:
    size: Its source size in bytes.
```

clear

```
clear(self) -> None

Remove both status lines before a log message is printed.
```

duration_text

```
duration_text(seconds: float) -> str

Format a duration compactly.

Args:
    seconds: Duration in seconds.

Returns:
    Text such as ``2d03h``, ``5h12m`` or ``42m``.
```

Prompt (mediatools.core.prompt)

```
Interactive yes/no and text questions.

With ``assume_yes=True`` every yes/no question is answered with yes
(used by ``--yes`` options); ``input_func`` can be replaced in tests.

Attributes:
    assume_yes: Answer every yes/no question with yes.
    input: Function used to read an answer.
```

__init__

```
__init__(self, assume_yes: bool = False, input_func: Callable[[str], str] = input)

Configure the prompt.

Args:
    assume_yes: Answer every yes/no question with yes.
    input_func: Function that shows a prompt and returns the answer.
```

yes_no

```
yes_no(self, question: str, default: bool = False) -> bool

Ask a yes/no question until a valid answer is given.

Args:
    question: Question text without the ``[y/N]`` suffix.
    default: Result for empty input or end of input.

Returns:
    True for yes, False for no.
```

text

```
text(self, question: str, default: str = '') -> str

Ask for free text.

Args:
    question: Question text.
    default: Result for empty input or end of input.

Returns:
    The stripped answer or ``default``.
```

RunControl (mediatools.core.runcontrol)

```
Lets ``mediatools stop`` ask a running job to end after its current file.

The running job writes its process ID to ``<data>/state/<name>.pid``.
``mediatools stop`` checks that this process is alive and then creates
``<data>/state/<name>.stop``; the job notices the file within
:attr:`POLL_SECONDS`, finishes the current file and ends - exactly like
the first Ctrl+C, but independent of which window has the keyboard
focus. Both files are removed when the job ends; a stop file found at
start (left by a crash) is removed as well. A second job of the same
name cannot start while the first one is alive (:meth:`start` raises
:class:`RuntimeError`); a PID file of a crashed job does not block.

``mediatools reload`` creates ``<data>/state/<name>.restart`` the same
way: the job switches to a newly installed release after its current
file (see :class:`SelfRestart`). It hands its registration over
(:meth:`handover`) instead of removing it, so a watching ``attach``
sees one continuous job with the same PID.

The PID file also names the log file, and the job writes its progress
lines to ``<data>/state/<name>.status``; ``mediatools attach`` shows
both. With ``background`` the job moves into the background when the
real work begins (:meth:`begin_work`, after the confirmation): it forks,
the terminal gets its prompt back, and the job continues in a new
session, independent of the terminal - closing the window does not end
it. Its terminal output goes to ``<data>/state/<name>.console``. The
optional sleep lock is taken in the process that does the work.

Attributes:
    name: Job name, e.g. ``shrink``.
    pid_file: Path of the PID file (PID and log file path).
    stop_file: Path of the stop-request file.
    restart_file: Path of the restart-request file.
    status_file: Path of the progress status file.
    console_file: Terminal output of a job in the background.
    last_poll: Monotonic time of the last check for the stop file.
    background: Move into the background in :meth:`begin_work`.
    inhibitor: Sleep lock held while the work runs, or None.
    log_path: Log file of the job, or None.
```

__init__

```
__init__(
    self,
    name: str,
    data: DataDir | None = None,
    background: bool = False,
    inhibitor: SleepInhibitor | None = None,
    log_path: str | None = None,
)

Locate the control files.

Args:
    name: Job name, e.g. ``shrink``.
    data: Data directory; the default one if None.
    background: Move into the background in :meth:`begin_work`.
    inhibitor: Sleep lock held while the work runs, or None.
    log_path: Log file of the job, or None.
```

start

```
start(self) -> None

Register the current process and drop a stale stop request.

Raises:
    RuntimeError: If another job of this name is still running.
```

_register

```
_register(self) -> None

Write PID and log file path of the current process.
```

begin_work

```
begin_work(self) -> None

Enter the work phase: background (if set), then the sleep lock.
```

detach

```
detach(self) -> None

Continue in a background process; the calling process exits.

The parent prints how to follow and stop the job, then ends at
once with ``os._exit(0)`` - without cleanup, which belongs to the
child. The child starts a new session (no controlling terminal, so
closing the window sends it no SIGHUP), reads from /dev/null,
writes its terminal output to :attr:`console_file` and registers
its own PID.
```

finish

```
finish(self) -> None

Unregister the current process and release the sleep lock.
```

handover

```
handover(self) -> None

Prepare the switch to a new release: keep PID and status files.

The sleep lock is released (the new process takes its own); stop
and restart requests are consumed.
```

restart_requested

```
restart_requested(self) -> bool

Tell whether ``mediatools reload`` asked for a switch.

Returns:
    True if the restart file exists.
```

clear_restart

```
clear_restart(self) -> None

Drop a restart request that cannot be served.
```

stop_requested

```
stop_requested(self, force: bool = False) -> bool

Tell whether a stop was requested, checking at most every POLL_SECONDS.

Args:
    force: Check now, ignoring the poll interval.

Returns:
    True if the stop file exists.
```

active_pid

```
active_pid(self) -> int | None

Return the process ID of the running job, if it is alive.

Returns:
    PID, or None if no job of this name runs.
```

active_log

```
active_log(self) -> str | None

Return the log file of the running job.

Returns:
    Path from the PID file, or None if no job runs or none is named.
```

request_stop

```
request_stop(self) -> int | None

Ask the running job to stop after its current file.

Returns:
    PID of the job that was asked, or None if none runs (then no
    stop file is left behind that would stop the next run).
```

request_restart

```
request_restart(self) -> int | None

Ask the running job to switch to the installed release after its current file.

Returns:
    PID of the job that was asked, or None if none runs (then no
    request file is left behind).
```

_remove

```
_remove(path: str) -> None

Delete a file if it exists.

Args:
    path: File to delete.
```

RunLog (mediatools.core.runlog)

```
Logger that writes to stdout and to a time-stamped log file.

The terminal shows plain messages; the file additionally contains
timestamp and level of every line, and always the DEBUG details
(e.g. ffmpeg commands, database records) that the terminal shows only
with ``verbose``. The first two lines name the log
file and the running version with its process ID, so every log tells
which release produced it.

Example:
    >>> run = RunLog("shrink")
    >>> run.logger.info("started")

Attributes:
    datadir: Data directory in which the log file is created.
    path: Absolute path of the log file.
    logger: The configured :class:`logging.Logger`.
```

`__init__`

```
__init__(
    self,
    name: str,
    verbose: bool = False,
    datadir: DataDir | None = None,
    continue_path: str | None = None,
)

Create the logger and its two handlers.

Args:
    name: Command name; used in the logger and file name.
    verbose: If True, DEBUG messages are logged as well.
    datadir: Data directory; the default one if None.
    continue_path: Existing log file to append to (a job that
        switched to a new release); None creates a new file.
```

`close`

```
close(self) -> None

Flush and close all handlers (important in tests).
```

`SelfRestart (mediatools.core.selfrestart)`

```
Restarts the current command with the installed release, in the same process.

Used by ``mediatools shrink`` when ``mediatools reload`` (or ``mediatools
update``) asks a running job to switch to a newly installed release.
The switch happens only between two files: the job finishes its current
file, closes its result table and then calls :meth:`execute`, which
replaces the process image with ``python -m mediatools`` of the same
interpreter (``os.execv``). The process keeps its PID, session, terminal
or console file and environment, so ``mediatools attach`` and ``stop``
keep working and the sleep lock (bound to the PID) is taken over.

The new process runs the same command line plus ``--foreground`` (it is
already in the background), ``-y`` (the user confirmed the run) and
``--continue-log`` (it appends to the same log file). It plans again;
every file finished so far is recorded in the databases, so it continues
with the files left. Before the switch, :meth:`installed_version` starts
the installed release once with ``--version``; a release that does not
answer is not switched to.

Attributes:
    argv: Command-line arguments of the running command (without the
        program name).
    log_path: Log file the new process continues.
    python: Interpreter of the installed release.
```

`__init__`

```
__init__(self, argv: list[str], log_path: str, python: str | None = None) -> None

Store what the new process needs.

Args:
    argv: Arguments of the running command, without the program name.
    log_path: Log file to continue.
    python: Interpreter; the running one if None.
```

`command`

```
command(self) -> list[str]

Build the command line of the new process.

Returns:
    Argument vector for :func:`os.execv` (element 0 is the program).
```

installed_version

```
installed_version(self) -> str | None

Ask the installed release for its version.

Returns:
    Version string, e.g. ``1.2.11``, or None if it does not start or
    answers unexpectedly.
```

execute

```
execute(self) -> None

Replace the current process by the new one; returns only on failure.

Raises:
    OSError: If the program cannot be started.
```

SleepInhibitor (mediatools.core.sleepinhibitor)

```
Hold a systemd sleep/idle inhibitor lock for the lifetime of a job.

Runs ``systemd-inhibit --what=sleep:idle --mode=block tail
--pid=<own pid> -f /dev/null`` in its own session: the lock lives as
long as the helper; ``tail --pid`` ends it at the latest when this
process dies, even after SIGKILL, so no lock is left behind. Ctrl+C in
the terminal does not reach the helper (own session). If systemd-inhibit
is missing or refuses the lock, the job runs without it and a warning
is logged.

Example:
    >>> lock = SleepInhibitor("mediatools shrink")
    >>> lock.start()
    >>> lock.stop()

Attributes:
    why: Reason shown by ``systemd-inhibit --list``.
    log: Optional logger.
    proc: The helper process while the lock is held, else None.
```

__init__

```
__init__(
    self,
    why: str,
    log: logging.Logger | None = None,
    inhibit: str = 'systemd-inhibit',
) -> None

Configure the lock.

Args:
    why: Reason shown by ``systemd-inhibit --list``.
    log: Optional logger.
    inhibit: Name or path of the systemd-inhibit program.
```

start

```
start(self) -> bool

Take the lock.

Returns:
    True if the lock is held.
```

stop

```
stop(self) -> None

Release the lock (no-op if none is held).
```

_warn

```
_warn(self, message: str) -> None

Log a warning if a logger is set.

Args:
    message: Warning text.
```

VersionInfo (mediatools.core.versioninfo)

```
Describes which mediatools is running.

An installed release lives in a ``site-packages`` directory. Code that is
imported straight from a clone - through ``PYTHONPATH`` or an editable
``pip install -e`` - lives in ``<clone>/src/mediatools`` next to the
clone's ``pyproject.toml``. ``--version`` names the checkout in that case,
so a development state is never mistaken for a release.

Attributes:
    package_dir: Directory of the imported ``mediatools`` package.
    version: Version number of the imported code.
```

__init__

```
__init__(self, package_dir: str | None = None, version: str = '1.2.12')

Locate the imported package.

Args:
    package_dir: Package directory; the imported one if None.
    version: Version number; the imported one by default.
```

checkout

```
checkout(self) -> str | None

Return the clone the code is imported from, if any.

Returns:
    Path of the source checkout, or None for an installed package.
```

describe

```
describe(self) -> str

Return the text printed by ``mediatools --version``.

Returns:
    ``"1.2.0"`` for an installed release, otherwise the version with
    the path of the source checkout.
```

TreeWalker (mediatools.core.walker)

```
Deterministic :func:`os.walk` that never enters reserved directories.

Skipped are ``_flat``, ``newShrinkedFiles``, hidden directories (``.*``)
and any additional names given. Directory and file lists are sorted so
that runs are reproducible.

Attributes:
    skip: Directory names that are never entered.
```

__init__

```
__init__(self, extra_skip: Iterable[str] = ())

Build the skip set.

Args:
    extra_skip: Directory names to skip in addition to the defaults.
```

walk

```
walk(self, root: str) -> Iterator[tuple[str, list[str], list[str]]]

Walk a tree top-down.

Args:
    root: Directory to walk.

Yields:
    ``(dirpath, dirs, files)`` like :func:`os.walk`, sorted.
```

files

```
files(self, root: str) -> Iterator[str]

Walk a tree and return regular files only.

Args:
    root: Directory to walk.

Yields:
    Absolute paths of regular files; symbolic links are left out.
```

media_files

```
media_files(self, root: str, extensions: frozenset = frozenset(...)) -> Iterator[str]

Walk a tree and return files with a given extension.

Args:
    root: Directory to walk.
    extensions: Accepted lower-case extensions including the dot.

Yields:
    Absolute paths of matching regular files; empty files (0 bytes,
    e.g. triage placeholders) are left out.
```

17.3 mediatools.shrink

ShrinkDatabase (mediatools.shrink.database)

```
The shrink database file of one directory.

One database exists per directory that contains videos. A row means
"this file has been processed" - shrunk by ``mediatools shrink`` or an
earlier tool, or found to be HEVC already - regardless of the codec.

**Matching is by file name within the directory**, not by the stored
absolute path: the database describes the files of its own directory,
and the collection may have been moved or renamed since the row was
written (the stored path then still points to the old location). The
stored size is compared only to report rows whose file has changed
since (:meth:`status` returns ``changed``); such files still count as
processed, so nothing is encoded twice.

The file name is configurable (``[shrink] db_name``, default
``shrink.db``); ``mediatools rename-db`` renames databases created under
another name. Databases with an early schema (no PRIMARY KEY) are backed
up and migrated on first use.

Videos that could not be shrunk are kept apart in a second table, so
``files`` keeps its meaning ("processed") for every tool that reads it.
A failure counts only while the file has the recorded size; a changed
file is tried again. Older tools ignore the extra table.

Schema::

    CREATE TABLE files (path text NOT NULL PRIMARY KEY,
                        size integer, modified text)
    CREATE TABLE failures (name text NOT NULL PRIMARY KEY, size integer,
                           status text, reason text, at text)

Attributes:
    name: File name of the database.
    path: Absolute path of the database file.
    log: Optional logger.
    migrated_backup: Path of the backup made during a migration, if any.
```

__init__

```
__init__(self, directory: str, logger=None, name: str | None = None)

Bind the object to a directory.

Args:
    directory: Directory whose database is handled.
    logger: Optional :class:`logging.Logger`.
    name: Database file name; :attr:`NAME` if None.
```

mtime_text

```
mtime_text(path: str) -> str

Format the mtime of a file as stored in the database.

Args:
    path: File whose mtime is read.

Returns:
    Local time ``YYYY-MM-DD HH:MM:SS``.
```

_connect

```
_connect(self) -> Iterator[sqlite3.Connection]

Open the database for one transaction and always close it.

``with sqlite3.connect(...)`` alone only commits or rolls back; it
does not close the connection. This wrapper does both.

Yields:
    Open connection; the 30 s busy timeout covers a GUI holding a lock.
```

exists

```
exists(self) -> bool

Tell whether the database file exists.

Returns:
    True if the file exists.
```

ensure

```
ensure(self) -> None

Create the database or migrate an old schema; add the failures table.
```

_ensure_files

```
_ensure_files(self) -> None

Create the files table or migrate an old schema.
```

_migrate

```
_migrate(self) -> None

Back up the file and rebuild the table with a PRIMARY KEY.

Duplicate paths keep the most recently inserted row; rows without a
path are dropped.
```

index

```
index(self) -> dict[str, set[int]]

Return the recorded file names with their sizes (cached).

Returns:
    Mapping file name (last path component of the stored path) to
    the set of recorded sizes; empty if no database exists.
```

status

```
status(self, file_path: str) -> str

Classify a file against the database.

Args:
    file_path: Absolute path of a video in this directory.

Returns:
    ``done`` (name and size recorded), ``regrown`` (name recorded,
    but the file is now at least :attr:`GROWTH` times as large as
    every recorded size - typically an original restored over its
    shrunk version), ``changed`` (name recorded with another size;
    counts as processed) or ``todo``.
```

is_done

```
is_done(self, file_path: str) -> bool

Tell whether a file counts as processed.

Args:
    file_path: Absolute path of the video.

Returns:
    True if its name is recorded (see :meth:`status`); a ``regrown``
    file counts as processed here - the planner decides with a codec
    check whether it is encoded again.
```

record

```
record(self, file_path: str) -> None

Insert or update the row of a file with its current size and mtime.

Args:
    file_path: Absolute path of the video.
```

record_failure

```
record_failure(self, file_path: str, status: str, reason: str) -> None

Note that a file could not be shrunk (call :meth:`ensure` first).

Args:
    file_path: Absolute path of the video.
    status: ``FAILED`` or ``BROKEN``.
    reason: Short cause.
```

failure

```
failure(self, file_path: str) -> str | None

Return the recorded failure status of a file of unchanged size.

Args:
    file_path: Absolute path of a video in this directory.

Returns:
    ``FAILED`` or ``BROKEN`` if the file is recorded as failed with
    its current size, otherwise None.
```

failures

```
failures(self) -> dict[str, tuple[int, str]]

Return the recorded failures (cached).

Returns:
    Mapping file name to ``(size, status)``; empty if the database
    or its failures table does not exist.
```

clear_failure

```
clear_failure(self, file_path: str) -> None

Forget a recorded failure, e.g. after a successful retry.

Args:
    file_path: Absolute path of the video.
```

rows

```
rows(self) -> list[tuple]

Return all rows (for reports and tests).

Returns:
    List of ``(path, size, modified)``; empty if no database exists.
```

DatabaseRenamer (mediatools.shrink.dbrenamer)

```
Renames ``<old>`` and ``<old>.bak_*`` to ``<new>`` and ``<new>.bak_*``.

Used when the database file name changes, e.g. after setting
``[shrink] db_name``. Renaming within a directory keeps the inode, so
the content is untouched. A directory in which the new name already
exists is reported as a conflict and left alone; nothing is ever
overwritten.

Attributes:
    root: Absolute start directory.
    old: Current database file name.
    new: Target database file name.
    log: Logger.
    walker: Tree walker with the shared skip rules.
```

__init__

```
__init__(self, root: str, old: str, new: str, logger)

Configure a run.

Args:
    root: Start directory, processed recursively.
    old: Current database file name.
    new: Target database file name.
    logger: :class:`logging.Logger`.

Raises:
    ValueError: If a name contains a path separator or both are equal.
```

plan

```
plan(self) -> list[tuple[str, str, str]]

Find all files to rename.

Returns:
    ``(source, destination, status)`` with status ``rename`` or
    ``conflict`` (destination exists), sorted by source path.
```

apply

```
apply(self, plan: list[tuple[str, str, str]]) -> dict[str, int]

Rename the planned files; conflicts are skipped.

Args:
    plan: Result of :meth:`plan`.

Returns:
    Counters ``renamed``, ``conflict`` and ``error``.
```

HevcEncoder (mediatools.shrink.encoder)

```
Encodes a video with ffmpeg/libx265 and validates the result.

The output is accepted only if ffmpeg exits with 0, the file is not
empty, it is at least :attr:`MIN_RATIO` of the input's size, it contains
a video stream, its duration matches the input within
``max(TOL_ABS, duration * TOL_REL)`` seconds and - with a
:class:`DecodeCheck` set - the whole output decodes without a single
error (:meth:`DecodeCheck.verify`; time limit
``max(VERIFY_MIN, duration * VERIFY_FACTOR)``). A
suspiciously small output is typical of an input whose data could not
be decoded: the encoder then writes almost nothing but grey frames.

**CPU use.** libx265 runs its own thread pool, which by default uses
every logical CPU; ffmpeg's ``-threads`` does not limit it. Both the
decoder (``-threads``) and the x265 pool (``-x265-params pools=N``) are
therefore set to ``threads``, by default one less than the number of
logical CPUs, so that one CPU stays free. In addition ffmpeg runs with
the lowest CPU priority (``nice`` 19): it still uses every idle CPU,
but yields at once when the user works, so the desktop stays
responsive.

ffmpeg runs in its own session (process group): a Ctrl+C in the
terminal reaches only mediatools, which decides whether the current
encode finishes or is stopped with :meth:`abort`. Progress is read from
``-progress pipe:1``; ffmpeg's messages go to ``<dst>.ffmpeg.log``; the
file is deleted after a validated encode and kept otherwise (its last
lines are logged; the caller decides where it is kept).

Attributes:
    crf: libx265 constant rate factor.
    threads: Decoder threads and size of the x265 thread pool.
    niceness: Scheduling priority of ffmpeg (0 normal, 19 lowest).
    probe: :class:`MediaProbe` used for validation.
    log: Optional logger.
    decode: Trial decode of the output, or None (no decode check).
    ffmpeg: Name or path of the ffmpeg executable.
    process: Running ffmpeg process, if any.
    aborted: True after :meth:`abort` during the current encode.
```

__init__

```
__init__(
    self,
    crf: int = 28,
    threads: int | None = None,
    probe: MediaProbe | None = None,
    logger=None,
    ffmpeg: str = 'ffmpeg',
    niceness: int = 19,
    decode: DecodeCheck | None = None,
)

Configure the encoder.

Args:
    crf: libx265 constant rate factor (28 is the libx265 default).
    threads: Decoder threads and x265 pool size; None uses
        :meth:`default_threads`.
    probe: Probe for validation; a new one if None.
    logger: Optional :class:`logging.Logger`.
    ffmpeg: Name or path of the ffmpeg executable.
    niceness: Scheduling priority of ffmpeg (0 normal, 19 lowest).
    decode: Trial decode of the output, or None (no decode check).
```

default_threads

```
default_threads() -> int

Return the default thread count: logical CPUs minus one.

Returns:
    At least 1.
```

command

```
command(self, src: str, dst: str) -> list[str]

Build the ffmpeg argument list (no shell is involved).

Args:
    src: Input video.
    dst: Output file.

Returns:
    Argument list for :func:`subprocess.run`; prefixed with
    ``nice -n N`` when a niceness is set and ``nice`` is available.
```

encode

```
encode(
    self,
    src: str,
    dst: str,
    on_progress: Callable[[float], None] | None = None,
) -> bool

Encode a video and validate the result.

Args:
    src: Input video.
    dst: Output file (overwritten).
    on_progress: Called with the encoded fraction (0..1) of the file.

Returns:
    True only for a validated result; False on error or abort.
```

abort

```
abort(self) -> None

Stop a running encode (SIGTERM to ffmpeg's process group).
```

_log_tail

```
_log_tail(self, log_path: str, lines: int = 8) -> None

Log the last lines of ffmpeg's message file.

Args:
    log_path: ffmpeg message file.
    lines: Number of lines to log.
```

validate

```
validate(self, src: str, dst: str) -> bool

Check size, video stream, duration and decodability of the output.

Args:
    src: Input video.
    dst: Encoded output.

Returns:
    True if the output is at least :attr:`MIN_RATIO` of the input,
    has a video stream, a matching duration (or the duration of
    either file cannot be determined) and - with a decode check -
    decodes completely without errors.
```

_info

```
_info(self, *args) -> None

Log details at DEBUG level (log file only) if a logger is set.

Args:
    *args: Format string and arguments for :meth:`logging.Logger.debug`.
```

_error

```
_error(self, *args) -> None

Log at ERROR level if a logger is set.

Args:
    *args: Format string and arguments for :meth:`logging.Logger.error`.
```

FileResult (mediatools.shrink.fileresult)

```
Outcome of one video.

Attributes:
    path: Absolute path of the video.
    status: ``shrunk``, ``copy`` (review mode), ``HEVC`` (recorded
        without encoding), ``FAILED`` or ``BROKEN`` (data not decodable,
        set aside).
    source: Size before, in bytes.
    result: Size after, in bytes, or None (failed, HEVC, broken).
    length: Playing time in seconds, or None if unknown.
    took: Wall-clock seconds spent on the file.
```

__replace__

```
__replace__(self, **changes)
```

__init__

```
__init__(
    self,
    path: str,
    status: str,
    source: int,
    result: int | None = None,
    length: float | None = None,
    took: float = 0.0,
) -> None

Initialize self.  See help(type(self)) for accurate signature.
```

__repr__

```
__repr__(self)

Return repr(self).
```

__eq__

```
__eq__(self, other)

Return self==value.
```

ReplaceJournal (mediatools.shrink.journal)

```
Copies a validated output over its original with a journal marker.

**Crash safety.** The validated output is flushed to disk, then a
marker ``newShrinkedFiles/<name>.replace`` is written and flushed, and
the directory entry is flushed; only then is the original overwritten
(in place: the inode is kept, so ``_flat`` hard links see the new
content). The marker is removed after the copy has been flushed and
recorded. If the machine fails in between, :meth:`recover` finds the
marker next to the complete output in the next run and repeats the
copy before anything else.

Attributes:
    log: Logger.
```

__init__

```
__init__(self, logger) -> None

Store the logger.

Args:
    logger: :class:`logging.Logger` for recovery messages.
```

replace

```
replace(self, out: str, path: str, database: ShrinkDatabase) -> None

Copy a validated output over the original and record it.

Order: flush the output, write and flush the marker, flush the
directory entry, copy into the existing inode, flush, record, delete
output, delete marker. A crash at any point leaves either the
untouched original or the marker plus the complete validated output;
repeating the copy is harmless, so a crash after step 5 only causes
one more identical copy on the next run.

Args:
    out: Validated output in ``newShrinkedFiles/``.
    path: Original video.
    database: Database of its directory.
```

recover

```
recover(self, dirpath: str, database_of: Callable[[str], ShrinkDatabase]) -> int

Repeat replacements that a crash interrupted.

Args:
    dirpath: Directory whose ``newShrinkedFiles`` is checked.
    database_of: Returns the database of a directory.

Returns:
    Number of replacements completed.
```

fsync_file

```
fsync_file(path: str) -> None

Force a file's content to disk.

Args:
    path: File.
```

fsync_dir

```
fsync_dir(path: str) -> None

Force a directory's entries (file creations) to disk.

Args:
    path: Directory.
```

ShrinkOptions (mediatools.shrink.options)

```
What a :class:`Shrinker` run processes and how.

Attributes:
    keep_copies: Do not replace originals and record nothing; the shrunk
        copies stay in ``newShrinkedFiles/`` for review.
    extensions: Extensions to process (lower case, with dot).
    db_name: Database file name; None uses the default name.
    order: ``largest`` (default), ``smallest`` or ``path``.
    min_seconds: Leave out videos shorter than this; 0 processes all.
    retry_failed: Include videos recorded as failed (``FAILED`` or
        ``BROKEN``) in an earlier run; by default they are left out
        while their size is unchanged.
```

__post_init__

```
__post_init__(self) -> None

Validate the values.

Raises:
    ValueError: If ``order`` is unknown or ``min_seconds`` negative.
```

__replace__

```
__replace__(self, **changes)
```

__hash__

```
__hash__(self)

Return hash(self).
```

__init__

```
__init__(
    self,
    keep_copies: bool = False,
    extensions: frozenset = <factory>,
    db_name: str | None = None,
    order: str = 'largest',
    min_seconds: float = 0.0,
    retry_failed: bool = False,
) -> None

Initialize self.  See help(type(self)) for accurate signature.
```

__repr__

```
__repr__(self)

Return repr(self).
```

__eq__

```
__eq__(self, other)

Return self==value.
```

__setattr__

```
__setattr__(self, name, value)

Implement setattr(self, name, value).
```

__delattr__

```
__delattr__(self, name)

Implement delattr(self, name).
```

ShrinkPlanner (mediatools.shrink.planner)

```
Walks the tree and lists the videos a shrink run encodes.

Per directory: finish replacements interrupted by a crash
(:class:`ReplaceJournal`), note leftover ``newShrinkedFiles/`` folders
of earlier aborted runs, and collect every video that is not yet
recorded in its directory's database. The database always has
precedence: a recorded file counts as processed - with one exception:
a file that is now clearly larger than recorded (``regrown``, e.g. an
original restored from a backup over its shrunk version) and whose
video is not HEVC is planned again. A video recorded as failed
(``FAILED`` or ``BROKEN``) with unchanged size is left out unless
``retry_failed`` is set. Then optionally drop
videos shorter than ``min_seconds``, set aside videos that are already
HEVC and sort the rest (``largest`` first, ``smallest`` first or walk
order ``path``).

Results go into the run's shared ``stats`` and ``found`` dictionaries
(see :class:`Shrinker`).

Attributes:
    root: Absolute start directory.
    log: Logger.
    opts: Run settings.
    probe: Probe for codec and duration checks.
    state: Run helpers; ``durations`` and ``codecs`` are used.
    database_of: Returns the database of a directory.
    journal: Completes interrupted replacements.
    stats: Shared counters ``skipped``, ``short``, ``recovered``,
        ``failed_before``, ``regrown``.
    found: Shared lists ``leftovers``, ``hevc`` and ``failed_before``.
```

__init__

```
__init__(
    self,
    root: str,
    logger,
    opts: ShrinkOptions,
    probe: MediaProbe,
    state: ShrinkState,
    database_of: Callable[[str], ShrinkDatabase],
    journal: ReplaceJournal,
    results: tuple[dict, dict],
) -> None

Bind the planner to a run.

Args:
    root: Absolute start directory.
    logger: :class:`logging.Logger`.
    opts: Run settings.
    probe: Probe for codec and duration checks.
    state: Run helpers (caches).
    database_of: Returns the database of a directory.
    journal: Completes interrupted replacements.
    results: The run's ``(stats, found)`` dictionaries.
```

plan

```
plan(self, stopped: Callable[[], bool]) -> list[tuple[str, int]]

Recover interrupted replacements and list the videos to encode.

Args:
    stopped: Returns True once the run should stop.

Returns:
    ``(path, size)`` pairs in processing order.
```

_candidate

```
_candidate(self, path: str) -> tuple[str, int] | None

Decide whether one file of the walk is a candidate.

Args:
    path: File in the tree.

Returns:
    ``(path, size)`` if it is to be encoded, else None (other
    extension, link, placeholder, recorded or recorded as failed).
```

_video_size

```
_video_size(self, path: str) -> int | None

Return the size of a regular, non-empty video with a planned extension.

Args:
    path: File in the tree.

Returns:
    Size in bytes, or None for another extension, a link, a missing
    file or a 0-byte triage placeholder.
```

_drop_short

```
_drop_short(self, todo: list[tuple[str, int]]) -> list[tuple[str, int]]

Remove videos shorter than ``min_seconds``.

Args:
    todo: Candidates.

Returns:
    Candidates with a known duration of at least ``min_seconds``
    (unknown durations are kept).
```

_set_aside_hevc

```
_set_aside_hevc(self, todo: list[tuple[str, int]]) -> list[tuple[str, int]]

Move videos that are already HEVC out of the encode list.

Args:
    todo: Candidates.

Returns:
    Candidates that need encoding; the others go to
    ``found["hevc"]``.
```

_checkpoint

```
_checkpoint(
    self,
    what: str,
    cache: DurationCache | None,
    number: int,
    total: int,
) -> None

Report planning progress and store newly probed values.

Saving every :attr:`CHECKPOINT` files keeps the probes of a long
first check even if the run is interrupted during planning.

Args:
    what: ``durations`` or ``codecs``.
    cache: The cache in use, or None.
    number: Files checked so far.
    total: Files to check.
```

_codec

```
_codec(self, path: str) -> str | None

Return a video's codec, from the cache if possible.

Args:
    path: Video file.

Returns:
    Codec name, or None if unknown.
```

_duration

```
_duration(self, path: str) -> float | None

Return a video's duration, from the cache if possible.

Args:
    path: Video file.

Returns:
    Seconds, or None if unknown.
```

_note_leftover

```
_note_leftover(self, dirpath: str) -> None

Remember a leftover work folder for the plan summary.

Args:
    dirpath: Directory whose ``newShrinkedFiles`` is checked.
```

ProgressReport (mediatools.shrink.progress)

```
Reports which videos are recorded in their shrink database and which not.

Strictly read-only: no database is created and nothing is changed.
Matching follows :meth:`ShrinkDatabase.status` (file name within the
directory), so databases written at an earlier location of the
collection are evaluated correctly. Four lists are written to
``<data>/reports/``: ``progress_done``, ``progress_changed`` (recorded,
size differs - counts as processed), ``progress_regrown`` (recorded,
but at least 20 % larger again - ``shrink`` encodes it again unless it
is HEVC) and ``progress_todo``. Regrown videos count towards the bytes
still to do.

Attributes:
    root: Absolute start directory.
    log: Logger.
    datadir: Data directory for the reports.
    extensions: Extensions that are counted.
    db_name: Database file name (None: default name).
    throughput: Measured encoding speed for the time estimate.
    lists: Paths per status ``done``, ``changed``, ``regrown``, ``todo``.
    todo_bytes: Total size of the videos still to do (todo and regrown).
    databases: Number of databases found.
    orphan_rows: Database rows without a matching file.
```

__init__

```
__init__(
    self,
    root: str,
    logger,
    datadir: DataDir | None = None,
    extensions: frozenset = frozenset(...),
    db_name: str | None = None,
    throughput: ThroughputLog | None = None,
)

Configure the report.

Args:
    root: Start directory.
    logger: :class:`logging.Logger`.
    datadir: Data directory; the default one if None.
    extensions: Extensions that are counted.
    db_name: Database file name; the default name if None.
    throughput: Speed measurements; loaded from the data directory if None.
```

scan

```
scan(self) -> None

Fill :attr:`lists` by walking the tree.
```

write

```
write(self) -> dict[str, str]

Write the four lists to the reports directory.

Returns:
    Report path per status.
```

run

```
run(self) -> int

Scan, write the lists and log a summary with a time estimate.

Returns:
    Always 0.
```

ResultTable (mediatools.shrink.resulttable)

```
Write one boxed table row per finished video.

Fixed-width columns hold the figures; the last column holds the path
relative to the shrink root and takes the remaining width. A longer
path is wrapped inside its cell at ``/`` boundaries, so the row grows in
height instead of breaking the frame. Rows are separated by horizontal
rules::

    ┌──────┬────────┬──────────┬──────────┬──────────┬──────────┬──────────┬────────┬──────────┐
    │   no │ end    │   source │   result │    ratio │   length │     took │ status │ file     │
    ├──────┼────────┼──────────┼──────────┼──────────┼──────────┼──────────┼────────┼──────────┤
    │    1 │ 06:54  │ 2860 MB  │  510 MB  │   17.8 % │ 44.3 min │ 27.9 min │ shrunk │ a/b/     │
    │      │        │          │          │          │          │          │        │ clip.mp4 │
    └──────┴────────┴──────────┴──────────┴──────────┴──────────┴──────────┴────────┴──────────┘

The terminal copy uses the terminal width; the report file uses
:attr:`REPORT_WIDTH` and is written row by row and flushed, so it is
complete up to the last finished file even after a crash.

Attributes:
    root: Shrink root; paths are shown relative to it.
    report: Path of the report file.
    log: Logger for the terminal and log file copy.
    width: Total width of the terminal copy.
    count: Number of rows written so far.
```

__init__

```
__init__(self, root: str, report: str, log: logging.Logger, width: int = 120) -> None

Create the table; the header is written with the first row.

Args:
    root: Shrink root.
    report: Path of the report file (created on the first row).
    log: Logger for the terminal and log file copy.
    width: Total width of the terminal copy.
```

add

```
add(self, item: FileResult) -> None

Write the row of one finished video.

Args:
    item: Outcome of the video.
```

finish

```
finish(self) -> None

Close the frame (only if rows were written since the last call).
```

_write

```
_write(self, lines: list[str]) -> None

Append lines to the report file and flush them.

Args:
    lines: Lines without line break.
```

file_width

```
file_width(self, width: int) -> int

Width of the file column for a total table width.

Args:
    width: Total width of the table.

Returns:
    Characters available for the path (at least :attr:`MIN_FILE`).
```

head

```
head(self, width: int) -> list[str]

Opening rule, column titles and the rule below them.

Args:
    width: Total width of the table.

Returns:
    Three lines.
```

row

```
row(self, cells: list[str], path: str, width: int) -> list[str]

Lines of one row; the path is wrapped inside its cell.

Args:
    cells: Texts of the fixed columns.
    path: Relative path of the video.
    width: Total width of the table.

Returns:
    One line per path piece; figures only on the first.
```

rule

```
rule(self, left: str, cross: str, right: str, width: int) -> str

Horizontal rule matching the columns.

Args:
    left: Left corner or tee.
    cross: Crossing between columns.
    right: Right corner or tee.
    width: Total width of the table.

Returns:
    The rule.
```

_line

```
_line(self, cells: list[str], path: str, width: int) -> str

One framed text line.

Args:
    cells: Texts of the fixed columns.
    path: Text of the file column.
    width: Total width of the table.

Returns:
    The line.
```

wrap

```
wrap(path: str, width: int) -> list[str]

Split a path into pieces of at most ``width`` characters.

Breaks after a ``/`` or a space where possible; only a single word
longer than ``width`` is cut hard.

Args:
    path: Relative path.
    width: Maximum piece length.

Returns:
    At least one piece.
```

cells

```
cells(self, number: int, item: FileResult) -> list[str]

Texts of the fixed columns.

Args:
    number: Row number.
    item: Outcome of the video.

Returns:
    One text per entry of :attr:`COLUMNS`.
```

mb

```
mb(size: int | None) -> str

Format a size in MB (10^6 bytes).

Args:
    size: Size in bytes, or None.

Returns:
    Text such as ``2860 MB`` or ``-``.
```

minutes

```
minutes(seconds: float | None) -> str

Format a duration in minutes.

Args:
    seconds: Duration in seconds, or None.

Returns:
    Text such as ``44.3 min`` or ``-``.
```

Shrinker (mediatools.shrink.shrinker)

```
Shrinks the videos below a directory and keeps their file names.

A run has two phases:

1. **Plan** - :class:`ShrinkPlanner`: finish replacements interrupted
   by a crash, note leftover ``newShrinkedFiles/`` folders, collect the
   videos not yet recorded (and not recorded as failed, unless
   ``retry_failed``), drop short ones, set aside HEVC, sort.
2. **Encode** - per file: already HEVC -> record only; otherwise
   a trial decode (:class:`DecodeCheck`, if set) sets aside videos whose
   data cannot be decoded (status ``BROKEN``, never encoded, never
   recorded, original untouched);
   :class:`HevcEncoder` writes to ``newShrinkedFiles/``; only a validated
   result replaces the original (:class:`ReplaceJournal`: in place,
   crash-safe, the inode is kept, so ``_flat`` hard links see the new
   content) and is recorded. A file that fails (``FAILED``) or cannot be
   decoded (``BROKEN``) is recorded as failed in its directory's
   database and left out of later runs while its size is unchanged;
   ffmpeg's complete message file of a failed encode is kept in
   ``ffmpeg_logs``.

**Stopping.** The first Ctrl+C (or Ctrl+Z), or ``mediatools stop`` from
another terminal (see :class:`RunControl`), lets the current file finish
and then stops; the second Ctrl+C, SIGTERM or SIGHUP (terminal closed)
aborts the current encode at once - its partial output is deleted, the
original is untouched. Signals never interrupt the replacement itself.

**Crash safety.** See :class:`ReplaceJournal`.

**New release during a run.** ``mediatools reload`` (also sent by
``mediatools update``) asks the job to switch to the installed release.
Between two files - never during an encode or a replacement - the job
checks that the installed release starts (``--version``), closes its
table and summary and returns :attr:`RESTART`; the caller then replaces
the process (:class:`SelfRestart`). A release that does not start, or
the same version, is not switched to and the run continues.

**Leftovers.** ``newShrinkedFiles/`` folders that remain after recovery
hold only partial outputs of aborted runs. They are listed in the plan
summary and deleted only after the start question is confirmed.

With ``keep_copies=True`` the originals are not touched and nothing is
recorded; the shrunk copies stay in ``newShrinkedFiles/`` for review.

Attributes:
    root: Absolute start directory.
    log: Logger.
    opts: What is processed and how (:class:`ShrinkOptions`).
    encoder: Encoder for new files.
    state: Persistent helpers: ``durations`` (for ``min_seconds``) and
        ``codecs`` (HEVC check of the plan) - None probes without caching -
        ``throughput`` (speed and size ratio for the estimates) and
        ``decode`` (trial decode before encoding; None skips it) and
        ``ffmpeg_logs`` (folder for the ffmpeg messages of failed
        encodes; None deletes them) and ``restart`` (switch to a new
        release on request; None ignores requests).
    planner: Planning phase; holds the probe and the replacement
        journal.
    control: Stop requests from ``mediatools stop``, or None.
    databases: Open databases per directory.
    flags: ``stop`` (finish the current file, then stop), ``abort``
        (stop the current file at once) and ``restart`` (switch to the
        installed release before the next file).
    display: Progress display of the encode phase.
    stats: Counters ``shrunk``, ``hevc``, ``skipped``, ``short``,
        ``failed``, ``broken``, ``failed_before`` (left out: recorded as
        failed), ``regrown`` (recorded, but larger again: planned),
        ``copies``, ``recovered``.
    found: ``failed`` - paths that could not be encoded; ``broken`` -
        paths whose data could not be decoded; ``failed_before`` - paths
        left out as recorded failures; ``leftovers`` -
        ``(folder, files, bytes)`` of leftover work folders; ``hevc`` -
        ``(path, size)`` of planned videos that are already HEVC.
```

__init__

```
__init__(
    self,
    root: str,
    logger,
    opts: ShrinkOptions | None = None,
    encoder: HevcEncoder | None = None,
    probe: MediaProbe | None = None,
    durations: DurationCache | None = None,
    throughput: ThroughputLog | None = None,
    display: ProgressBar | None = None,
    codecs: CodecCache | None = None,
)

Configure a run.

Args:
    root: Start directory, processed recursively.
    logger: :class:`logging.Logger` for progress and results.
    opts: Run settings; defaults if None.
    encoder: Encoder; a default :class:`HevcEncoder` if None.
    probe: Probe; a new :class:`MediaProbe` if None.
    durations: Duration cache; None probes without caching.
    throughput: Speed measurements; a default one if None.
    display: Progress display; created for the encode phase if None.
    codecs: Codec cache for the plan; None probes without caching.
```

install_signal_handlers

```
install_signal_handlers(self) -> None

Route SIGINT, SIGTERM, SIGHUP and SIGTSTP to :meth:`on_signal`.
```

on_signal

```
on_signal(self, signum: int, _frame: object = None) -> None

First Ctrl+C: stop after this file; second Ctrl+C, SIGTERM, SIGHUP: abort now.

Only flags are set and ffmpeg is told to stop; no exception is
raised, so a running replacement always completes. After SIGHUP the
terminal is gone, so the progress line is switched off.

Args:
    signum: Number of the received signal.
```

database

```
database(self, dirpath: str) -> ShrinkDatabase

Return the (cached) database of a directory.

Args:
    dirpath: Directory.

Returns:
    Its :class:`ShrinkDatabase`.
```

plan

```
plan(self) -> list[tuple[str, int]]

Recover interrupted replacements and list the videos to encode.

Returns:
    ``(path, size)`` pairs in processing order (see
    :class:`ShrinkPlanner`).
```

run

```
run(self, confirm: Callable[[], bool] | None = None) -> int

Plan, report, ask and encode.

Args:
    confirm: Called after the plan summary; encoding starts only if
        it returns True. None starts without asking.

Returns:
    0 on success or if declined, 1 if stopped by the user, 2 if a
    file failed, :attr:`RESTART` if the caller is to switch to the
    installed release.
```

_run

```
_run(self, confirm: Callable[[], bool] | None) -> int

Body of :meth:`run`.

Args:
    confirm: See :meth:`run`.

Returns:
    See :meth:`run`.
```

_report_plan

```
_report_plan(self, todo: list[tuple[str, int]], total: int, rate: float | None) -> None

Log the plan summary with time and size estimates and leftovers.

Args:
    todo: Planned videos.
    total: Their total size in bytes.
    rate: Measured speed in source bytes per second, or None.
```

_attach_bar_filter

```
_attach_bar_filter(self) -> None

Clear the progress line before each console log message.
```

_process

```
_process(self, path: str, size: int) -> bool

Handle one planned video.

Args:
    path: Absolute path of the video.
    size: Its size when planned.

Returns:
    True if an encode was attempted, False if the file needed none
    (removed, recorded meanwhile, or HEVC by now - e.g. a hard link
    of a file shrunk earlier in this run) or its data cannot be
    decoded.
```

_broken

```
_broken(self, path: str, size: int) -> bool

Set a video aside if a trial decode finds its data unusable.

Such a file is not encoded and stays untouched; it is recorded as
failed, so later runs leave it out while its size is unchanged.

Args:
    path: Video file.
    size: Its size.

Returns:
    True if the video was set aside.
```

_record_failure

```
_record_failure(self, path: str, status: str, reason: str) -> None

Note a failed video in its directory's database (not in review mode).

Args:
    path: Video file.
    status: ``FAILED`` or ``BROKEN``.
    reason: Short cause for the log and the database.
```

_keep_ffmpeg_log

```
_keep_ffmpeg_log(self, out: str, path: str) -> None

Keep ffmpeg's complete message file of a failed encode.

Args:
    out: Output path of the encode; its messages are in
        ``<out>.ffmpeg.log``.
    path: Original video (for the name and the log line).
```

_record_hevc

```
_record_hevc(self) -> None

Record the videos set aside as HEVC during planning.
```

_record_one_hevc

```
_record_one_hevc(self, path: str) -> None

Record a video that is already HEVC, without encoding it.

Args:
    path: Video file.
```

_encode_and_replace

```
_encode_and_replace(
    self,
    path: str,
    size: int,
    out: str,
    database: ShrinkDatabase,
) -> None

Encode one video and, if valid, replace the original.

Args:
    path: Original video.
    size: Its size.
    out: Output path in ``newShrinkedFiles/``.
    database: Database of its directory.
```

_result

```
_result(self, item: FileResult) -> None

Add a row to the result table, if one is set.

Args:
    item: Outcome of one video.
```

_show

```
_show(self, name: str, fraction: float, size: int) -> None

Forward encoder progress to the display.

Args:
    name: Current file name.
    fraction: Encoded fraction of the file.
    size: Its size in bytes.
```

_poll_stop

```
_poll_stop(self, force: bool = False) -> None

Turn a request of ``mediatools stop`` into a stop after this file.

Args:
    force: Check the stop file now, ignoring the poll interval.
```

_poll_restart

```
_poll_restart(self) -> None

Turn a request of ``mediatools reload`` into a switch before the next file.
```

_mark_stopping

```
_mark_stopping(self, message: str) -> None

Set the stop flag and show it in the log and on the progress line.

Args:
    message: Log message.
```

_remove_leftovers

```
_remove_leftovers(self) -> None

Delete the leftover work folders noted during planning.
```

_size_or_none

```
_size_or_none(path: str) -> int | None

Return a file's size.

Args:
    path: File.

Returns:
    Size in bytes, or None if it cannot be read.
```

_remove_quietly

```
_remove_quietly(path: str) -> None

Delete a file if it exists.

Args:
    path: File to delete.
```

_finish

```
_finish(self) -> None

Log the summary.
```

ShrinkState (mediatools.shrink.shrinkstate)

```
Helpers of a shrink run, kept together in one attribute.

Attributes:
    durations: Cache of playing times (for ``min_seconds``), or None.
    codecs: Cache of video codecs (HEVC check of the plan), or None.
    throughput: Measured speed and size ratio for the estimates.
    results: Result table of finished videos, or None.
    decode: Trial decode that sets aside undecodable videos, or None
        (no check).
    ffmpeg_logs: Folder for ffmpeg's messages of failed encodes, or
        None (they are deleted).
    restart: Switch to a newly installed release on request, or None
        (requests are ignored).
```

ThroughputLog (mediatools.shrink.throughput)

```
Accumulates encoding speed and size reduction of finished files.

Speed (source bytes per second of encoding) and size ratio (output
bytes per source byte) depend on CPU, codec settings and material;
measured on the user's own machine they are the best basis for the
estimates shown before a run. Stored as JSON in ``<data>/state/``:
``{"bytes", "seconds"}`` for the speed and ``{"ratio_src", "ratio_out",
"files"}`` for the size ratio (the ratio keys were added in 1.2.1, so
older files carry only the speed).

Attributes:
    path: Location of the JSON file.
    bytes: Accumulated source bytes.
    seconds: Accumulated encoding seconds.
    ratio_src: Source bytes of the files whose output size is known.
    ratio_out: Output bytes of those files.
    files: Number of those files.
```

__init__

```
__init__(self, path: str | None = None)

Load the accumulated values.

Args:
    path: JSON file; ``<data>/state/shrink_throughput.json`` if None.
```

rate

```
rate(self) -> float | None

Return the measured speed.

Returns:
    Source bytes per second, or None without measurements.
```

ratio

```
ratio(self) -> float | None

Return the measured size ratio.

Returns:
    Output bytes per source byte, or None without measurements.
```

add

```
add(self, size: int, seconds: float, out_size: int | None = None) -> None

Add one encoded file and save.

Args:
    size: Source size in bytes.
    seconds: Encoding time in seconds.
    out_size: Output size in bytes; None leaves the ratio unchanged.
```

duration_text

```
duration_text(seconds: float) -> str

Format a duration for humans.

Args:
    seconds: Duration in seconds.

Returns:
    Text such as ``3 d 4 h``, ``5 h 12 min`` or ``42 min``.
```

17.4 mediatools.triage

CandidateFinder (mediatools.triage.candidates)

```
Yields videos whose duration lies in ``[min_s, max_s]``.

Iterating is lazy (a generator); :meth:`collect` runs the whole search
first so that the number of candidates is known before the review
starts. ``_flat`` and ``newShrinkedFiles`` are never entered and empty
placeholder files are skipped, hence every video is offered once under
its original name. Durations are taken from a :class:`DurationCache`
when one is given, so repeated scans are fast.

Attributes:
    root: Start directory.
    extensions: Accepted extensions.
    min_s: Lower duration threshold in seconds.
    max_s: Upper duration threshold in seconds.
    kept: Kept list or None (then nothing is filtered).
    probe: Probe for the durations.
    log: Optional logger.
    cache: Optional duration cache.
    scanned: Number of videos looked at so far.
```

__init__

```
__init__(
    self,
    root: str,
    extensions: frozenset,
    min_s: float,
    max_s: float,
    kept: KeptList | None = None,
    probe: MediaProbe | None = None,
    logger=None,
    cache: DurationCache | None = None,
)

Configure the search.

Args:
    root: Start directory.
    extensions: Accepted extensions.
    min_s: Lower duration threshold in seconds.
    max_s: Upper duration threshold in seconds.
    kept: Files on this list are skipped; None disables the filter.
    probe: Probe; a new one if None.
    logger: Optional :class:`logging.Logger`.
    cache: Duration cache; None probes every file.
```

`__iter__`

```
__iter__(self) -> Iterator[tuple[str, float]]

Walk the tree and probe each video.

Yields:
    ``(path, duration_seconds)`` of every matching video.
```

`_duration`

```
_duration(self, path: str) -> float | None

Return the duration, from the cache if possible.

Args:
    path: Video file.

Returns:
    Duration in seconds, or None if ffprobe cannot tell.
```

`collect`

```
collect(self) -> list[tuple[str, float]]

Run the complete search before the review starts.

Returns:
    All matching ``(path, duration)`` pairs in walk order; the cache
    (if any) is saved afterwards.
```

`KeptList` (`mediatools.triage.keptlist`)

```
Remembers kept files across triage sessions.

Stored as TSV ``path<TAB>size<TAB>mtime`` (mtime as integer Unix time).
A file counts as kept only while all three values still match, so a
file that changes (e.g. is shrunk) is offered again.

Attributes:
    path: Location of the TSV file.
```

`__init__`

```
__init__(self, path: str)

Bind the list to a file.

Args:
    path: Location of the TSV file (created on first write).
```

key

```
key(file_path: str) -> str

Build the identity line of a file.

Args:
    file_path: File to describe.

Returns:
    ``path<TAB>size<TAB>mtime``.
```

load

```
load(self) -> set[str]

Read the list once and cache it.

Returns:
    Set of identity lines.
```

contains

```
contains(self, file_path: str) -> bool

Tell whether a file is on the list with unchanged size and mtime.

Args:
    file_path: File to look up.

Returns:
    True if the identity line is present.
```

add

```
add(self, paths: Iterable[str]) -> None

Append files to the list (duplicates are ignored).

Args:
    paths: Files to remember.
```

FlatLinkResolver (mediatools.triage.linkresolver)

```
Finds the names of a file inside the ``_flat`` directories above it.

``mediatools flatten`` creates hard links in ``<unit>/_flat/``. Removing
only the original name would not free any space, because the link keeps
the content alive. This resolver returns those sibling names so that
triage can remove them together with the original.

Only ``_flat`` directories of ancestors up to ``root`` are searched;
hard links elsewhere (e.g. created by a deduplication tool in another
unit) are deliberately left alone and reported via the link count.

Attributes:
    root: Absolute directory above which no ``_flat`` is searched.
```

__init__

```
__init__(self, root: str)

Bind the resolver to a search root.

Args:
    root: Top directory of the triage run.
```

`_flat_index`

```
_flat_index(self, flat_dir: str) -> dict[tuple[int, int], list[str]]

Return (and cache) the inode index of one ``_flat`` directory.

Args:
    flat_dir: Path of a ``_flat`` directory.

Returns:
    Mapping ``(st_dev, st_ino)`` to the paths with that inode.
```

`flat_names`

```
flat_names(self, path: str) -> list[str]

List all ``_flat`` names that share the inode of a file.

Args:
    path: Original file.

Returns:
    Paths in ``_flat`` directories of the file's ancestors.
```

`forget`

```
forget(self, path: str) -> None

Drop a removed name from the cache.

Args:
    path: Path that no longer exists.
```

`PlayerSession` (`mediatools.triage.player`)

```
Starts a batch of player processes and classifies their state.

Closing a player window ends its process with exit code 0. After the
user confirms, :meth:`classify` sorts every file into

* ``closed`` - process ended normally after the start window: discard
* ``open`` - process still running: keep
* ``errors`` - process ended during the start window or with an error
  code: keep (protects against single-instance handover and crashes)

VLC is started with ``--no-one-instance --no-playlist-enqueue`` so that
each file gets its own process even if "allow only one instance" is set
in the VLC preferences.

Attributes:
    argv: Player command prefix; the file path is appended.
    start_window: Seconds after start in which an exit counts as error.
    stagger: Pause between two player starts in seconds.
    items: One dict per started file: ``path``, ``proc``, ``early``.
```

`__init__`

```
__init__(
    self,
    player: str = 'vlc',
    audio: bool = False,
    start_window: float = 3.0,
    stagger: float = 0.3,
)

Configure the session.

Args:
    player: Player command, e.g. ``vlc`` or ``mpv``.
    audio: Play with sound.
    start_window: Seconds after start in which an exit is an error.
    stagger: Pause between two player starts in seconds.
```

build_argv

```
build_argv(player: str, audio: bool) -> list[str]

Build the player command with safe defaults for VLC and mpv.

Args:
    player: Player command; may contain arguments
        (e.g. ``flatpak run org.videolan.VLC``).
    audio: Play with sound.

Returns:
    Argument list without the file path.
```

open

```
open(self, paths: list[str]) -> None

Start one process per file, then flag processes that ended at once.

Args:
    paths: Files to show.
```

classify

```
classify(self) -> tuple[list[str], list[str], list[tuple[str, int]]]

Sort the files by the state of their player process.

Returns:
    ``(closed, still_open, errors)``; ``errors`` holds
    ``(path, exit_code)`` pairs.
```

close_all

```
close_all(self, grace: float = 5.0) -> None

Terminate remaining processes (SIGTERM, then SIGKILL).

Args:
    grace: Seconds to wait for a clean exit before SIGKILL.
```

FileRemover (mediatools.triage.remover)

```
Removes files either via ``gio trash`` (default) or permanently.

``gio trash`` follows the FreeDesktop trash specification; trashed files
can be listed with ``gio trash --list`` and restored with
``gio trash --restore <uri>``. Trashed files still occupy disk space
until the trash is emptied.

:meth:`discard` additionally leaves an empty placeholder under the
original name, so a downloader that skips existing file names does not
fetch the discarded video again.

Attributes:
    permanent: Delete instead of trashing.
    log: Optional logger.
    gio: Name or path of the gio executable.
```

__init__

```
__init__(self, permanent: bool = False, logger=None, gio: str = 'gio')

Configure the remover.

Args:
    permanent: Delete instead of trashing.
    logger: Optional :class:`logging.Logger`.
    gio: Name or path of the gio executable.
```

verb

```
verb(self) -> str

Human readable action.

Returns:
    ``delete permanently`` or ``move to trash``.
```

remove

```
remove(self, path: str) -> bool

Remove one file.

Args:
    path: File to remove.

Returns:
    True on success, False if the removal failed (logged).
```

discard

```
discard(self, path: str, placeholder: bool = True) -> bool

Remove a file and optionally leave a 0-byte placeholder.

The placeholder is created exclusively (``open(..., "x")``) right
after the removal, so an existing file is never overwritten. Its
modification time is the current time; ``sync`` therefore neither
copies it over the full file on a backup (``--min-size=1``) nor
restores the old file over it (``-u``).

Args:
    path: File to remove.
    placeholder: Create the empty file under the same name.

Returns:
    True if the file was removed (and, if requested, the placeholder
    created); False otherwise (logged).
```

Triage (mediatools.triage.triage)

```
Interactive batch review: close a window to discard that video.

Before the first batch, :meth:`announce` reports how many videos match
and roughly how long the review will take, and asks whether to start.

Workflow per batch:

1. open ``batch`` candidates, one player process each
2. the user **closes** the windows of the videos to **remove** and leaves
   the windows of the videos to **keep** open, then presses Enter
3. summary (remove / keep / player error) and confirmation
4. removal of each discarded file **together with its ``_flat`` hard
   links** (otherwise no space would be freed); by default an empty
   placeholder keeps the original name, so a downloader does not fetch
   the video again; kept files go to the kept list; remaining players
   are closed

Attributes:
    root: Absolute start directory.
    log: Logger.
    finder: Iterable of ``(path, duration)`` candidates.
    player: Player session.
    remover: File remover.
    kept: Kept list.
    batch: Videos per batch.
    prompt: Prompt for the questions.
    links: Resolver for ``_flat`` names.
    batch_no: Number of the current batch.
    placeholder: Leave a 0-byte file under the name of a removed video.
    discarded_log: TSV file that records every removed video.
```

__init__

```
__init__(
    self,
    root: str,
    logger,
    finder: Iterable[tuple[str, float]],
    player: PlayerSession,
    remover: FileRemover,
    kept: KeptList,
    batch: int = 10,
    prompt: Prompt | None = None,
    placeholder: bool = True,
    discarded_log: str | None = None,
)

Wire the collaborators.

Args:
    root: Start directory of the run.
    logger: :class:`logging.Logger`.
    finder: Candidates, usually a :class:`CandidateFinder`.
    player: Player session (or a test double).
    remover: File remover.
    kept: Kept list.
    batch: Videos per batch.
    prompt: Prompt; a default :class:`Prompt` if None.
    placeholder: Leave an empty file under the name of a removed video.
    discarded_log: TSV of removed videos; ``<data>/state/triage_discarded.tsv``
        if None.
```

install_signal_handlers

```
install_signal_handlers(self) -> None

Ctrl+C / SIGTERM close all players and remove nothing.
```

_on_signal

```
_on_signal(self, signum, _frame) -> None

Close players and exit with code 130.

Args:
    signum: Number of the received signal.
    _frame: Current stack frame (unused).
```

announce

```
announce(
    self,
    candidates: list[tuple[str, float]],
    scanned: int,
    ask: bool = True,
) -> bool

Report size and estimated duration of the session and ask to start.

The estimate assumes that each batch is watched once through its
longest clip plus :attr:`SECONDS_PER_DECISION` for the decision.

Args:
    candidates: All matching ``(path, duration)`` pairs.
    scanned: Number of videos examined in the search.
    ask: Ask before starting; False only reports.

Returns:
    True if the review should start.
```

run

```
run(self) -> int

Review all candidates batch by batch.

Returns:
    Always 0 (quitting is a normal end).
```

review

```
review(self, batch: list[tuple[str, float]]) -> bool

Handle one batch.

Args:
    batch: ``(path, duration)`` pairs of this batch.

Returns:
    False if the user quits the session, otherwise True.
```

_discard

```
_discard(self, path: str, flat_names: list[str]) -> None

Remove the ``_flat`` links and the video, record the removal.

Args:
    path: Original video.
    flat_names: Its ``_flat`` hard links.
```

_summary

```
_summary(
    self,
    plan: dict[str, list[str]],
    still_open: list[str],
    errors: list[tuple[str, int]],
) -> None

Log what will happen with each file of the batch.

Args:
    plan: Closed files mapped to their ``_flat`` names.
    still_open: Files whose player is still running.
    errors: ``(path, exit_code)`` of players that failed.
```

_other_links

```
_other_links(path: str, flat: list[str]) -> int

Count hard links outside the original and its ``_flat`` names.

Args:
    path: Original file.
    flat: Its ``_flat`` names.

Returns:
    Number of further names that would keep the data alive.
```

kept_list

```
kept_list(cls, datadir: DataDir | None = None) -> KeptList

Return the default kept list ``<data>/state/triage_kept.tsv``.

Args:
    datadir: Data directory; the default one if None.

Returns:
    The kept list object.
```

17.5 mediatools.flatten

FlatIndex (mediatools.flatten.flatindex)

```
Maps flat link names to their source path relative to the unit.

File format (UTF-8, tab separated, one line per link)::

    <link name><TAB><relative source path>

The index makes every link traceable regardless of how strongly its
name had to be shortened. Independently of the index, the original of a
link can always be found by inode: ``find <unit> -samefile <link>``.

Attributes:
    path: Location of ``_index.tsv``.
    entries: Mapping link name to relative source path.
```

__init__

```
__init__(self, flat_dir: str)

Bind the index to a ``_flat`` directory.

Args:
    flat_dir: Path of the ``_flat`` directory.
```

load

```
load(self) -> 'FlatIndex'

Read the index if it exists.

Returns:
    The object itself, to allow ``FlatIndex(d).load()``.
```

save

```
save(self) -> None

Write the index atomically (temporary file + rename).
```

by_source

```
by_source(self) -> dict[str, str]

Return the inverse mapping.

Returns:
    Mapping relative source path to link name.
```

Flattener (mediatools.flatten.flattener)

```
Creates ``<unit>/_flat/`` with one hard link per media file of the unit.

Terms:
    root: the directory the command is started in (master level).
    unit: every direct sub-directory of ``root`` (e.g. one project or one source).

Guarantees:
    * Originals are never modified, moved or deleted.
    * Only ``_flat`` directories are written to.
    * A hard link costs a directory entry, not the file's size.
    * Re-runs only add what is missing (idempotent); a source that is
      already linked under any name is recognised by its inode.

Workflow: :meth:`analyse` builds a :class:`FlatPlan`; the caller shows
the summary, saves the plan and calls :meth:`apply` after
confirmation. :meth:`undo` reverts the links of a saved plan.

Attributes:
    root: Absolute master directory.
    log: Logger.
    extensions: Extensions that are linked.
    abbreviate: Use folder abbreviations from naming level 1 on.
    prune: Plan removal of links whose source is gone.
    abbreviations: Folder abbreviations (None: namer defaults).
    walker: Tree walker with the shared skip rules.
```

__init__

```
__init__(
    self,
    root: str,
    logger,
    extensions: frozenset = frozenset(...),
    abbreviate: bool = True,
    prune: bool = False,
    abbreviations: dict[str, str] | None = None,
)

Configure a run.

Args:
    root: Master directory; its sub-directories are the units.
    logger: :class:`logging.Logger`.
    extensions: Extensions that are linked.
    abbreviate: Use folder abbreviations from naming level 1 on.
    prune: Plan removal of links whose source is gone.
    abbreviations: Folder abbreviations; the namer's defaults if None.
```

units

```
units(self) -> list[str]

List the units below root.

Returns:
    Absolute paths of direct sub-directories, sorted; hidden
    directories, reserved names and symbolic links are left out.
```

sources

```
sources(self, unit: str) -> Iterator[tuple[str, list[str]]]

List the media files of a unit.

Args:
    unit: Unit directory.

Yields:
    ``(absolute_path, path_components_below_unit)``; files on another
    file system are logged and skipped (hard links cannot cross).
```

name_limit

```
name_limit(path: str) -> int

Return the maximum file name length of the file system.

Args:
    path: Any path on the file system.

Returns:
    Limit in bytes; eCryptfs with encrypted names reports 143.
```

analyse

```
analyse(self) -> FlatPlan

Compare every unit with its ``_flat`` directory.

Returns:
    Plan with one entry per source (and per orphan with ``prune``).
```

_analyse_unit

```
_analyse_unit(self, unit: str, plan: FlatPlan) -> None

Add the entries of one unit to the plan.

Args:
    unit: Unit directory.
    plan: Plan that receives the entries.
```

_existing

```
_existing(flat: str) -> dict[str, tuple[int, int]]

Read the current content of a ``_flat`` directory.

Args:
    flat: Path of the ``_flat`` directory.

Returns:
    Mapping link name to ``(st_dev, st_ino)``; empty if missing.
```

apply

```
apply(self, plan: FlatPlan) -> dict[str, int]

Execute the ``link``, ``relink`` and ``prune`` entries.

Args:
    plan: Plan from :meth:`analyse`.

Returns:
    Counters ``link``, ``relink``, ``prune`` and ``error``.
```

undo

```
undo(self, plan: FlatPlan) -> int

Remove the links created by a saved plan.

Only links that still point to the same inode as their source are
removed, so nothing that changed in between is touched.

Args:
    plan: Plan loaded with :meth:`FlatPlan.load`.

Returns:
    Number of removed links.
```

FlatNamer (mediatools.flatten.namer)

```
Encodes the relative path of a file into one file name.

``posts/2024/Videos/clip_01.mp4`` becomes
``posts__2024__Videos__clip_01.mp4``. The separator ``__`` is used
because it hardly ever occurs in real folder names.

If the result exceeds ``limit`` bytes (UTF-8), increasingly compact
levels are tried:

====== ============================================================
Level  Rule
====== ============================================================
0      full path, ``__`` separated
1      configured folder names abbreviated, ``<id> <date> <time>``
       folders compacted to ``<id>-<YYYYMMDD>T<hhmmss>``
2      level 1 plus the front of the file stem cut; the tail
       (usually the media id) is kept, ``~`` marks the cut
3      directory part replaced by an 8-digit hash, file stem cut
====== ============================================================

The exact origin of every link is always recorded in ``_index.tsv``,
so even level 3 names stay traceable.

Attributes:
    limit: Maximum name length in bytes.
    abbrev: Folder name abbreviations used from level 1 on.
```

__init__

```
__init__(self, limit: int = 255, abbrev: dict | None = None)

Configure the namer.

Args:
    limit: Maximum name length in bytes (``statvfs().f_namemax``).
    abbrev: Folder abbreviations; None selects the defaults, an
        empty dict disables abbreviation.
```

nbytes

```
nbytes(name: str) -> int

Return the length of a name as stored by the file system.

Args:
    name: File name.

Returns:
    Length in bytes (file-system encoding, normally UTF-8).
```

fits

```
fits(self, name: str) -> bool

Tell whether a name respects the byte limit.

Args:
    name: File name.

Returns:
    True if the name is not longer than :attr:`limit` bytes.
```

compact

```
compact(self, segment: str) -> str

Return the level-1 form of one directory name.

Args:
    segment: Directory name.

Returns:
    Abbreviation, compacted id/date form, or the unchanged name.
```

short_hash

```
short_hash(text: str, length: int = 8) -> str

Return a stable short hash of a text.

Args:
    text: Text to hash.
    length: Number of hex digits.

Returns:
    Prefix of the SHA-1 hex digest.
```

_cut_stem

```
_cut_stem(self, prefix: str, filename: str) -> str | None

Shorten the file stem from the front until prefix and name fit.

Args:
    prefix: Directory part including the trailing separator.
    filename: Original file name.

Returns:
    Shortened name, or None if fewer than 8 bytes of stem would remain.
```

name

```
name(self, parts: list[str]) -> tuple[str, int] | None

Build the flat name for a relative path.

Args:
    parts: Path components below the unit; the last element is the
        file name.

Returns:
    ``(flat_name, level)``, or None if no level fits the limit.
```

disambiguate

```
disambiguate(self, name: str, relpath: str) -> str

Make a name unique by inserting ``~<hash6>`` before the extension.

Args:
    name: Name that is already taken.
    relpath: Relative source path; makes the hash stable per file.

Returns:
    New name within the byte limit.
```

FlatPlan (mediatools.flatten.plan)

```
Collection of :class:`PlanEntry` with summary and TSV persistence.

The TSV file written by :meth:`save` is also the input for
``mediatools flatten --undo PLAN``.

Attributes:
    entries: All planned or executed actions.
```

add

```
add(self, entry: PlanEntry) -> None

Append an entry.

Args:
    entry: Action to add.
```

by_action

```
by_action(self, action: str) -> list[PlanEntry]

Select entries of one kind.

Args:
    action: Action name, e.g. ``link``.

Returns:
    Matching entries in plan order.
```

counts

```
counts(self) -> dict[str, int]

Count entries per action.

Returns:
    Mapping action name to number of entries.
```

level_counts

```
level_counts(self) -> dict[int, int]

Count new and renewed links per naming level.

Returns:
    Mapping level to number of ``link``/``relink`` entries.
```

save

```
save(self, path: str) -> None

Write the plan as TSV.

Args:
    path: Target file.
```

load

```
load(cls, path: str) -> 'FlatPlan'

Read a plan written by :meth:`save`.

Args:
    path: Plan file.

Returns:
    The reconstructed plan.
```

__replace__

```
__replace__(self, **changes)
```

__init__

```
__init__(self, entries: list[PlanEntry] = <factory>) -> None

Initialize self.  See help(type(self)) for accurate signature.
```

__repr__

```
__repr__(self)

Return repr(self).
```

__eq__

```
__eq__(self, other)

Return self==value.
```

PlanEntry (mediatools.flatten.planentry)

```
One action of a flatten run.

Attributes:
    action: ``link`` (new), ``relink`` (source replaced, link renewed),
        ``prune`` (orphan link removed), ``ok`` (already present) or
        ``skip`` (cannot be linked, see ``note``).
    unit: Name of the unit directory.
    link: Absolute path of the link in ``_flat``.
    source: Absolute path of the source file.
    level: Naming level 0..3 of the link name.
    note: Free text, e.g. the reason for ``skip``.
```

__replace__

```
__replace__(self, **changes)
```

__init__

```
__init__(
    self,
    action: str,
    unit: str,
    link: str,
    source: str,
    level: int = 0,
    note: str = '',
) -> None

Initialize self.  See help(type(self)) for accurate signature.
```

`__repr__`

```
__repr__(self)

Return repr(self).
```

`__eq__`

```
__eq__(self, other)

Return self==value.
```

17.6 `mediatools.sync`

FsCheck (`mediatools.sync.fscheck`)

```
Answers "can this directory hold hard links?" and related questions.

The file-system type is read from ``/proc/self/mounts`` (longest
matching mount point). The decisive test is practical: a temporary file
is created and hard-linked in the target directory.

Attributes:
    mounts_file: Mount table to read.
```

`__init__`

```
__init__(self, mounts_file: str = '/proc/self/mounts')

Select the mount table.

Args:
    mounts_file: Mount table to read (replaceable in tests).
```

`mounts`

```
mounts(self) -> list[tuple[str, str, str]]

Read the mount table.

Returns:
    List of ``(device, mount point, fs type)``.
```

`mount_of`

```
mount_of(self, path: str) -> tuple[str, str, str] | None

Find the mount that contains a path.

Args:
    path: Any path.

Returns:
    ``(device, mount point, fs type)`` of the longest matching mount
    point, or None.
```

`fs_type`

```
fs_type(self, path: str) -> str | None

Return the file-system type of a path.

Args:
    path: Any path.

Returns:
    Type such as ``ext4``, or None if unknown.
```

`can_hardlink`

```
can_hardlink(directory: str) -> tuple[bool, str]

Create and hard-link a temporary file in a directory.

Args:
    directory: Directory to test (must exist).

Returns:
    ``(ok, explanation)``.
```

`free_bytes`

```
free_bytes(path: str) -> int

Return the free space available to the user.

Args:
    path: Any path on the file system.

Returns:
    Free bytes for unprivileged users.
```

`nested`

```
nested(first: str, second: str) -> bool

Tell whether one path lies inside the other (or both are equal).

Args:
    first: First path.
    second: Second path.

Returns:
    True if the paths are equal or one contains the other.
```

RsyncRunner (mediatools.sync.rsync_runner)

```
Builds and runs the rsync command used by ``mediatools sync``.

Options and why:

``-a``                 archive mode (recursion, times, permissions, symlinks)
``-H``                 hard links in the source stay hard links in the target
``-u``                 never overwrite a newer file on the receiving side
``--no-inc-recursive`` full file list first, so hard links are found before
                       data is copied (rsync manual recommendation)
``--min-size=1``       empty files are not transferred: a triage placeholder
                       never overwrites the full copy on the backup
``--exclude``          temporary shrink output and partial downloads

``--delete`` is deliberately **never** used: files that exist only in the
target stay untouched and are reported instead.

Attributes:
    source: Source directory with trailing slash.
    target: Target directory with trailing slash.
    rsync: Name or path of the rsync executable.
```

__init__

```
__init__(self, source: str, target: str, rsync: str = 'rsync')

Store the directories.

Args:
    source: Source directory.
    target: Target directory.
    rsync: Name or path of the rsync executable.
```

command

```
command(self, dry_run: bool, log_file: str | None = None) -> list[str]

Build the complete argument list.

Args:
    dry_run: Add ``--dry-run`` and itemized output.
    log_file: rsync log file for the real run.

Returns:
    Argument list for :func:`subprocess.run`.
```

dry_run

```
dry_run(self) -> dict[str, Any]

Run rsync with ``--dry-run`` and summarise the changes.

Returns:
    Result of :meth:`parse` plus ``returncode`` and ``stderr``.
```

parse

```
parse(itemized: str) -> dict[str, Any]

Count new files, updates, hard links and new directories.

Itemize codes (rsync manual): ``>f+++++++++`` new file, ``>f.s......``
changed file, ``hf`` hard link, ``cd+++++++++`` new directory.

Args:
    itemized: stdout of an rsync run with ``--itemize-changes``.

Returns:
    Counters ``new``, ``update``, ``hardlink``, ``newdir``, ``other``
    and up to ten example paths in ``samples_new``.
```

run

```
run(self, log_file: str | None = None) -> int

Run the real transfer with progress on the terminal.

Args:
    log_file: rsync log file.

Returns:
    Exit code of rsync (0 ok, 24 vanished source files).
```

Syncer (mediatools.sync.syncer)

```
Copies a media tree to a backup disk (or back) and reports leftovers.

Steps:

1. safety checks - rsync present, source and target not nested, target
   file system known to support hard links, practical hard-link test
2. dry run - summary of new files, updates, hard links
3. confirmation, then the real rsync run (see :class:`RsyncRunner`)
4. report of files that exist only in the target (:class:`TargetReport`)

The same command works in both directions: ``sync A B`` copies A to B,
``sync B A`` (or ``--reverse``) copies back. Because nothing is ever
deleted, a reverse run also restores files that were removed on purpose
in the other tree; the dry-run summary shows how many files would be
created.

Attributes:
    source: Absolute source directory.
    target: Absolute target directory.
    log: Logger.
    prompt: Prompt for the confirmation.
    datadir: Data directory for logs and reports.
    report: Write the target-only report after the transfer.
    all_duplicates: Include duplicate groups of the whole target.
    fs: File-system helper.
```

__init__

```
__init__(
    self,
    source: str,
    target: str,
    logger,
    prompt: Prompt | None = None,
    datadir: DataDir | None = None,
    report: bool = True,
    all_duplicates: bool = False,
    fs: FsCheck | None = None,
)

Configure the synchronisation.

Args:
    source: Source directory.
    target: Target directory (created if missing).
    logger: :class:`logging.Logger`.
    prompt: Prompt; a default one if None.
    datadir: Data directory; the default one if None.
    report: Write the target-only report after the transfer.
    all_duplicates: Include duplicate groups of the whole target.
    fs: File-system helper; a default one if None.
```

check

```
check(self) -> bool

Run all safety checks; the reason of a failure is logged.

Returns:
    True if the transfer may start.
```

run

```
run(self) -> int

Execute checks, dry run, transfer and report.

Returns:
    0 on success or when the user declined, 1 on error.
```

write_report

```
write_report(self) -> str

Analyse the target-only files and write the TSV report.

Returns:
    Path of the report file.
```

TargetPicker (mediatools.sync.target_picker)

```
Asks where the backup disk is mounted and remembers the answer.

Candidates are mounted file systems below ``/media``, ``/run/media`` and
``/mnt``. The chosen target directory is stored per source in
``<data>/state/sync_targets.json`` and offered as default next time.

Attributes:
    prompt: Prompt for the questions.
    fs: File-system helper.
    state: Path of the JSON file with remembered targets.
```

__init__

```
__init__(
    self,
    prompt: Prompt | None = None,
    fs: FsCheck | None = None,
    datadir: DataDir | None = None,
)

Wire the collaborators.

Args:
    prompt: Prompt; a default one if None.
    fs: File-system helper; a default one if None.
    datadir: Data directory; the default one if None.
```

candidates

```
candidates(self) -> list[tuple[str, str]]

List mounted external disks.

Returns:
    ``(mount point, fs type)`` pairs below the known prefixes.
```

_load

```
_load(self) -> dict

Read the remembered targets.

Returns:
    Mapping source path to target path; empty if none stored.
```

remember

```
remember(self, source: str, target: str) -> None

Store the target for a source.

Args:
    source: Source directory.
    target: Chosen target directory.
```

pick

```
pick(self, source: str) -> str | None

Ask for the target directory.

Args:
    source: Source directory (used for the remembered default).

Returns:
    Target directory, or None if the user aborted.
```

TargetReport (mediatools.sync.target_report)

```
Classifies files that exist in the target but not in the source.

Each target-only file gets one status:

``hardlinked``  another name of the same inode exists in the target;
                removing this name would free no space
``duplicate``   another file in the target has identical content
                (different inode) - removable without data loss, or
                could be hard-linked to save space
``unique``      the only copy - keep unless you know it is obsolete

Content comparison: size first, then SHA-256 of the first and last MiB,
then the full SHA-256 only for files that still match.

With ``all_duplicates=True`` the whole target is additionally scanned
for groups of identical files stored in different inodes (space that a
deduplication by hard links could reclaim).

Attributes:
    source: Absolute source directory.
    target: Absolute target directory.
    log: Optional logger.
```

__init__

```
__init__(self, source: str, target: str, logger=None)

Bind the report to source and target.

Args:
    source: Source directory.
    target: Target directory.
    logger: Optional :class:`logging.Logger`.
```

_inventory

```
_inventory(self, root: str) -> dict[str, os.stat_result]

Stat every regular file of a tree.

Args:
    root: Directory to scan; ``_flat`` is included on purpose.

Returns:
    Mapping relative path to stat result.
```

quick_hash

```
quick_hash(self, path: str) -> str

Hash size, first MiB and last MiB of a file (cached).

Args:
    path: File to hash.

Returns:
    SHA-256 hex digest.
```

full_hash

```
full_hash(self, path: str) -> str

Hash the whole file (cached).

Args:
    path: File to hash.

Returns:
    SHA-256 hex digest.
```

same_content

```
same_content(self, first: str, second: str) -> bool

Tell whether two files are byte-identical.

Args:
    first: First file.
    second: Second file.

Returns:
    True if quick hash and full hash are equal.
```

analyse

```
analyse(self, all_duplicates: bool = False) -> dict

Classify the target-only files.

Args:
    all_duplicates: Also collect duplicate groups of the whole target.

Returns:
    ``{"target_only": [(rel, size, status, same_as), ...],
    "dup_groups": [[rel, ...], ...]}``.
```

_duplicate_of

```
_duplicate_of(
    self,
    rel: str,
    st: os.stat_result,
    dst: dict,
    by_size: dict[int, list[str]],
) -> tuple[str, str]

Look for another inode with identical content.

Args:
    rel: Relative path of the target-only file.
    st: Its stat result.
    dst: Target inventory.
    by_size: Target files grouped by size.

Returns:
    ``("duplicate", other_rel)`` or ``("unique", "")``.
```

_dup_groups

```
_dup_groups(self, dst: dict, by_size: dict[int, list[str]]) -> list[list[str]]

Find groups of identical files stored in different inodes.

Args:
    dst: Target inventory.
    by_size: Target files grouped by size.

Returns:
    Sorted groups of relative paths (one representative per inode).
```

write

```
write(result: dict, path: str) -> dict

Write the TSV report.

Args:
    result: Output of :meth:`analyse`.
    path: Report file.

Returns:
    Counters per status, ``bytes_duplicate`` and ``dup_groups``.
```

17.7 mediatools.snapshot

RsnapshotConf (mediatools.snapshot.rsnapconf)

```
Writes ``<data root>/snapshot/rsnapshot.conf`` and calls rsnapshot.

The file is generated from :class:`SnapshotSettings` on every run, so the
INI file stays the single place to edit. Important fixed choices:

* ``no_create_root 1`` - rsnapshot refuses to work if the snapshot root
  is missing, i.e. if the disk is not mounted (otherwise it would fill
  the empty mount point on the system disk).
* ``sync_first 1`` - ``rsnapshot sync`` copies into ``.sync``; the level
  commands only rotate.
* ``link_dest 1`` and ``-aH --no-inc-recursive`` - unchanged files and
  hard links (``_flat`` views) are shared, never duplicated.
* ``--delete`` only affects ``.sync``; deleted files remain in older
  snapshots.

Attributes:
    settings: Validated settings.
    path: Location of the generated file.
    log_file: rsnapshot's own log file.
    lock_file: rsnapshot's lock file (prevents parallel runs).
```

__init__

```
__init__(self, settings: SnapshotSettings, datadir: DataDir | None = None)

Choose the file locations.

Args:
    settings: Validated settings.
    datadir: Data directory; the default one if None.
```

render

```
render(self) -> str

Build the configuration text (fields separated by tabs).

Returns:
    Content of ``rsnapshot.conf``.

Raises:
    SystemExit: If ``cp`` or ``rsync`` cannot be found.
```

write

```
write(self) -> str

Write the configuration file.

Returns:
    Its path.
```

command

```
command(self, *args: str) -> list[str]

Build an rsnapshot command line for the generated file.

Args:
    *args: rsnapshot arguments, e.g. ``"sync"``.

Returns:
    Argument list.
```

configtest

```
configtest(self) -> tuple[bool, str]

Run ``rsnapshot configtest``.

Returns:
    ``(ok, output)``.
```

run

```
run(self, *args: str) -> int

Run rsnapshot with output on the terminal.

Args:
    *args: rsnapshot arguments, e.g. ``"sync"`` or ``"alpha"``.

Returns:
    Exit code (0 ok, 1 fatal error, 2 warnings).
```

LevelSchedule (mediatools.snapshot.schedule)

```
Remembers when each higher level was last rotated.

The backup disk is not permanently attached, so fixed timer dates do
not fit. Instead every run rotates ``alpha`` and additionally each
higher level whose period (beta 7, gamma 30, delta 365 days) has passed
since its last successful rotation. rsnapshot requires the higher
levels to be rotated first, so :meth:`due` returns them from the least
to the most frequent.

Attributes:
    settings: Validated settings.
    path: JSON state file ``{level: ISO time}``.
```

`__init__`

```
__init__(self, settings: SnapshotSettings, datadir: DataDir | None = None)

Locate the state file.

Args:
    settings: Validated settings.
    datadir: Data directory; the default one if None.
```

`load`

```
load(self) -> dict[str, str]

Read the state file.

Returns:
    Mapping level to ISO time of the last rotation; empty if none.
```

`due`

```
due(self, now: datetime) -> list[str]

List the higher levels to rotate in this run.

Args:
    now: Current time.

Returns:
    Level names, least frequent first; ``alpha`` is not included.
```

`record`

```
record(self, level: str, when: datetime) -> None

Store the time of a successful rotation.

Args:
    level: Level name.
    when: Time of the rotation.
```

`SnapshotSettings` (`mediatools.snapshot.settings`)

```
All values needed by :class:`Snapshotter`, already checked.

Attributes:
    source: Absolute directory that is backed up.
    disk: Mount point of the backup disk.
    root: Absolute snapshot root on the disk.
    retain: ``(level, count)`` from the most to the least frequent level.
    exclude: rsync exclude patterns.
    max_deletions: Abort threshold for files removed from the mirror.
```

`from_config`

```
from_config(cls, config: Config) -> 'SnapshotSettings'

Read and validate the ``[snapshot]`` section.

Args:
    config: Loaded configuration.

Returns:
    The settings.

Raises:
    SystemExit: With an explanation of the expected value if
        something is missing or inconsistent.
```

mirror

```
mirror(self, base: str) -> str

Return where the source lives inside a snapshot directory.

rsnapshot is used with ``--relative`` and the backup destination
``localhost/``, so ``/home/u/coll`` ends up in
``<base>/localhost/home/u/coll``.

Args:
    base: Snapshot directory, e.g. ``<root>/.sync`` or ``<root>/alpha.0``.

Returns:
    Absolute path of the mirrored source inside ``base``.
```

__replace__

```
__replace__(self, **changes)
```

__hash__

```
__hash__(self)

Return hash(self).
```

__init__

```
__init__(
    self,
    source: str,
    disk: str,
    root: str,
    retain: tuple[tuple[str, int], ...],
    exclude: tuple[str, ...],
    max_deletions: int,
) -> None

Initialize self.  See help(type(self)) for accurate signature.
```

__repr__

```
__repr__(self)

Return repr(self).
```

__eq__

```
__eq__(self, other)

Return self==value.
```

__setattr__

```
__setattr__(self, name, value)

Implement setattr(self, name, value).
```

__delattr__

```
__delattr__(self, name)

Implement delattr(self, name).
```

Snapshotter (mediatools.snapshot.snapshotter)

```
Drives rsnapshot on a removable backup disk.

Commands (see the handbook for the procedure):

* :meth:`check`  - read-only checks of configuration, disk and tools.
* :meth:`adopt`  - one-time takeover of an existing mirror on the disk
  as the first snapshot, without copying data (``rename`` on the same
  file system), optionally kept permanently as ``stand0``.
* :meth:`init`   - create an empty snapshot root for a fresh start.
* :meth:`run`    - probe, sync with ``--delete`` into ``.sync`` and
  rotate the due levels.
* :meth:`status` - list the snapshots.

Every command that changes the disk first runs a :class:`SyncProbe`
and refuses to continue if the result is implausible.

Attributes:
    settings: Validated settings.
    log: Logger.
    prompt: Confirmation helper.
    conf: rsnapshot configuration.
    probe: Dry-run helper.
    schedule: Level schedule.
    fscheck: File-system helper.
    datadir: Data directory for reports.
    ismount: Mount-point test.
```

__init__

```
__init__(
    self,
    settings: SnapshotSettings,
    logger,
    prompt: Prompt,
    conf: RsnapshotConf | None = None,
    probe: SyncProbe | None = None,
    schedule: LevelSchedule | None = None,
    fscheck: FsCheck | None = None,
    datadir: DataDir | None = None,
    ismount: Callable[[str], bool] = ismount,
)

Wire the collaborators.

Args:
    settings: Validated settings.
    logger: :class:`logging.Logger`.
    prompt: Confirmation helper.
    conf: rsnapshot configuration; built from settings if None.
    probe: Dry-run helper; built from settings if None.
    schedule: Level schedule; built from settings if None.
    fscheck: File-system helper; a new one if None.
    datadir: Data directory; the default one if None.
    ismount: Mount-point test (replaceable in tests).
```

_path

```
_path(self, name: str) -> str

Return a path inside the snapshot root.

Args:
    name: Entry name, e.g. ``alpha.0``.

Returns:
    Absolute path.
```

latest_mirror

```
latest_mirror(self) -> str | None

Return the newest copy of the source inside the snapshot root.

Returns:
    Mirror inside ``.sync`` or ``alpha.0``; None if neither exists.
```

_write_list

```
_write_list(self, prefix: str, paths: list[str]) -> str

Write a list of relative paths to a report file.

Args:
    prefix: File name prefix.
    paths: Lines to write.

Returns:
    Path of the report.
```

_log_probe

```
_log_probe(self, result: dict[str, Any], label: str) -> str

Log a probe result and save the deletion list.

Args:
    result: Result of :meth:`SyncProbe.run`.
    label: Report name prefix.

Returns:
    Path of the deletion list.
```

_top

```
_top(directory: str) -> list[str]

List the first entries of a directory for orientation.

Args:
    directory: Directory to list.

Returns:
    Up to 12 sorted entry names (``/`` appended to directories).
```

check

```
check(self) -> bool

Check configuration, disk and tools; nothing is changed.

Returns:
    True if a run or an adoption may proceed.
```

probe_report

```
probe_report(self) -> bool

Explain the state of the snapshot root and probe the newest mirror.

Read-only; complements :meth:`check` and tells the user which
command is expected next.

Returns:
    True if the configuration is accepted and the newest mirror (if
    any) is plausibly a copy of the source.
```

_which

```
_which(tool: str) -> bool

Tell whether an executable is on the PATH.

Args:
    tool: Executable name.

Returns:
    True if found.
```

_configtest

```
_configtest(self) -> bool

Write the rsnapshot configuration and test it.

Returns:
    True if rsnapshot accepts it.
```

init

```
init(self) -> int

Create an empty snapshot root (fresh start, first run copies all).

Returns:
    0 on success, 1 if checks fail or the root already exists.
```

adopt

```
adopt(self, existing: str, keep_stand0: bool = True) -> int

Turn an existing mirror on the disk into the first snapshot.

Nothing is copied: the directory is renamed into the layout
rsnapshot expects (``.sync/localhost/<source path>``) and rotated to
``alpha.0``; ``stand0`` is a hard-link copy (``cp -al``) that is
never rotated away and therefore keeps files that a later sync
removes from the mirror.

Args:
    existing: Directory on the disk that is a copy of the source.
    keep_stand0: Create the permanent ``stand0`` snapshot.

Returns:
    0 on success, 1 if a check fails or the user declines.
```

_adoptable

```
_adoptable(self, existing: str) -> bool

Check location and state for an adoption.

Args:
    existing: Candidate directory.

Returns:
    True if the directory is on the disk and the snapshot root is
    empty or missing.
```

_matches

```
_matches(self, existing: str) -> bool

Probe whether a directory is a copy of the source.

Args:
    existing: Candidate directory.

Returns:
    True if the transfer volume is plausible; otherwise the expected
    and the found content are shown so the user can correct the path.
```

run

```
run(self, dry_run: bool = False, now: datetime | None = None) -> int

Probe, sync and rotate.

Args:
    dry_run: Only probe and report.
    now: Current time (replaceable in tests).

Returns:
    0 on success, 1 on refusal or error.
```

_root_ready

```
_root_ready(self) -> bool

Check that the snapshot root exists.

Returns:
    True if it exists; otherwise the next step is explained.
```

_probe_acceptable

```
_probe_acceptable(self) -> bool

Probe the sync and apply the space and deletion limits.

Returns:
    True if the run may proceed.
```

_sync_and_rotate

```
_sync_and_rotate(self, now: datetime) -> int

Run ``rsnapshot sync`` and rotate the due levels.

Args:
    now: Current time.

Returns:
    0 on success, 1 if rsnapshot reported a fatal error.
```

_inode

```
_inode(self, name: str) -> int | None

Return the inode of an entry in the snapshot root.

Args:
    name: Entry name.

Returns:
    Inode number, or None if it does not exist.
```

status

```
status(self) -> int

Log the snapshots, their dates and the level schedule.

Returns:
    0, or 1 if the snapshot root is not available.
```

SyncProbe (mediatools.snapshot.syncprobe)

```
Compares the source with a mirror without changing anything.

The dry run uses the same rsync options as rsnapshot (``-aH --delete
--no-inc-recursive`` plus the excludes) and reports what a real sync
would do. It answers the two questions that decide whether a run is
safe: *is this mirror really a copy of this source?* (transfer volume)
and *how much would disappear from it?* (deletions).

Attributes:
    settings: Validated settings.
    rsync: rsync executable.
```

__init__

```
__init__(self, settings: SnapshotSettings, rsync: str = 'rsync')

Configure the probe.

Args:
    settings: Validated settings.
    rsync: rsync executable.
```

command

```
command(self, mirror: str) -> list[str]

Build the dry-run command line.

Args:
    mirror: Existing copy of the source.

Returns:
    Argument list.
```

run

```
run(self, mirror: str) -> dict[str, Any]

Run the dry run against a mirror.

Args:
    mirror: Existing copy of the source.

Returns:
    See :meth:`parse`, plus ``returncode`` and ``stderr``.
```

parse

```
parse(cls, output: str) -> dict[str, Any]

Evaluate itemized output and statistics.

Args:
    output: stdout of the dry run.

Returns:
    ``deleted`` (list of relative paths), ``new`` and ``changed``
    (file counts), ``total_bytes`` (source size) and
    ``transfer_bytes`` (data a real sync would write).
```

looks_like_copy

```
looks_like_copy(result: dict[str, Any], tolerance: float = 0.25) -> bool

Tell whether the mirror is plausibly a copy of the source.

If more than ``tolerance`` of the source volume would have to be
transferred, the mirror is most likely a different directory (wrong
path or wrong level) and must not be adopted.

Args:
    result: Result of :meth:`run`.
    tolerance: Accepted share of data to transfer.

Returns:
    True if the transfer volume is within the tolerance.
```

SystemdTrigger (mediatools.snapshot.trigger)

```
Creates ``~/.config/systemd/user/mediatools-snapshot.service``.

The service is ``WantedBy`` the mount unit of the backup disk, so the
user's systemd instance starts it whenever the desktop mounts the disk.
Mount units of all mounted file systems are visible to the user
instance; :meth:`mount_unit_known` verifies that for the actual disk
before anything is installed. The run uses ``-y``: it never asks, and it
aborts instead if the deletion limit would be exceeded.

Attributes:
    disk: Mount point of the backup disk.
    launcher: Command that starts mediatools.
    unit_dir: Directory of user units.
```

__init__

```
__init__(self, disk: str, launcher: str, unit_dir: str | None = None)

Configure the trigger.

Args:
    disk: Mount point of the backup disk.
    launcher: Absolute path of the ``mediatools`` launcher.
    unit_dir: Directory of user units; ``~/.config/systemd/user`` if None.
```

mount_unit

```
mount_unit(self) -> str

Return the systemd mount unit name of the disk.

Returns:
    Name such as ``media-user-BACKUP.mount``.

Raises:
    SystemExit: If ``systemd-escape`` is not available.
```

mount_unit_known

```
mount_unit_known(self) -> bool

Ask the user's systemd instance whether the mount unit is active.

Returns:
    True if ``systemctl --user`` reports the unit as active.
```

render

```
render(self) -> str

Build the unit file.

Returns:
    Content of the service file.
```

install

```
install(self) -> str

Write the unit file.

Returns:
    Its path.
```

17.8 mediatools.update

ReleaseUpdater (mediatools.update.updater)

```
Apply delivered patch archives with the bundled ``apply-release.sh``.

The shell script is shipped inside the package, so every installed
release carries it. It finds the patch archives in the download folder,
asks up to which version to apply, runs the git-flow hotfix steps in the
clone, installs the result and tidies the download folder (see the
script header and the developer documentation).

Example:
    >>> ReleaseUpdater(["--list"]).run()

Attributes:
    args: Arguments passed on to the script.
    script: Path of the bundled script.
```

__init__

```
__init__(self, args: list[str]) -> None

Prepare the call.

Args:
    args: Arguments for the script, e.g. ``["--list"]``.
```

command

```
command(self) -> list[str]

Build the command line.

Returns:
    ``bash <script> <args>``.

Raises:
    FileNotFoundError: If bash or the bundled script is missing.
```

run

```
run(self) -> int

Run the script in this terminal (it may ask questions).

Returns:
    Exit code of the script.
```